TRƯỜNG ĐẠI HỌC ĐỒNG THÁP

**KHOA SƯ PHẠM TOÁN - TIN**



0020410419 – NGUYỄN HOÀNG NAM

**ĐỒ ÁN MÔN HỌC 2**

# XÂY DỰNG WEBSITE ĐẶT LỊCH KHÁM BỆNH TẠI PHÒNG KHÁM HOÀNG NAM

Mã số đề tài: IT20A2_035

NGÀNH: KHOA HỌC MÁY TÍNH

LỚP: ĐHCNTT20A

*Đồng Tháp, ngày 29 tháng 12 năm 2023*

TRƯỜNG ĐẠI HỌC ĐỒNG THÁP

**KHOA SƯ PHẠM TOÁN - TIN**



0020410419 – NGUYỄN HOÀNG NAM

**ĐỒ ÁN MÔN HỌC 2**

# XÂY DỰNG WEBSITE ĐẶT LỊCH KHÁM BỆNH TẠI PHÒNG KHÁM HOÀNG NAM

Mã số đề tài: IT20A2_035

NGÀNH: KHOA HỌC MÁY TÍNH

LỚP: ĐHCNTT20A

**Giảng viên hướng dẫn: …….**

*Đồng Tháp, tháng 12 năm 2023*

# Lời cảm ơn

Trước tiên, em xin gửi lời cảm ơn đến quý Thầy Cô bộ môn Công Nghệ Thông Tin, khoa Sư phạm Toán – Tin đã trang bị cho em những kiến thức, kỹ năng cần thiết để hoàn thành báo cáo đồ án môn học 2 này.

Do kiến thức bản thân còn hạn chế nên trong quá trình thực hiện, hoàn thiện đề tài còn nhiều thiếu sót khi tìm hiểu, đánh giá và trình bày về đề tài. Rất mong sự quan tâm và đóng góp của Thầy Cô và để đề tài được đầy đủ và hoàn chỉnh hơn.

Em xin chân thành cảm ơn!

Đồng Tháp, ngày 29 tháng 12 năm 2023

Nguyễn Hoàng Nam

# Lời dẫn

Trong thời kì đầy thách thức và phát triển của cuộc sống ngày nay, không chỉ sự thành công về vật chất mà còn sự khỏe mạnh toàn diện được đặt lên hàng đầu. Sức khỏe không chỉ là chìa khóa để thực hiện những kế hoạch hàng ngày mà còn là nền tảng của tất cả những thành công lớn hơn trong cuộc sống.

Chúng ta đang sống trong một thế giới năng động và phát triển nhanh chóng, đặc biệt trong lĩnh vực y tế. Cùng với sự gia tăng về ý thức về sức khỏe, nhu cầu về một hệ thống chăm sóc sức khỏe hiện đại và tiện lợi càng trở nên quan trọng.

Ở đây, muốn nói về một cải tiến quan trọng đó là hệ thống đặt lịch trực tuyến. Đối với bệnh nhân, việc có thể đặt lịch từ nhà mình mà không cần phải đối mặt với thủ tục rườm rà tại bệnh viện mang lại sự thoải mái và tiết kiệm thời gian không giới hạn.

Bằng cách sử dụng ứng dụng đặt lịch khám trực tuyến, mọi người có khả năng chủ động quản lý sức khỏe của mình một cách linh hoạt. Hãy tưởng tượng việc bạn có thể chọn lịch hẹn phù hợp từ góc sofa ấm cúng của nhà mình chỉ với vài thao tác trên điện thoại thông minh. Điều này không chỉ giúp bệnh nhân tiết kiệm thời gian và công sức mà còn mang lại sự chuyên nghiệp và thuận tiện cho phòng khám. Hệ thống đặt lịch trực tuyến không chỉ là cải tiến kỹ thuật mà còn là sự đồng lòng của chúng ta với sứ mệnh cung cấp dịch vụ chăm sóc sức khỏe tốt nhất và dễ tiếp cận nhất cho mọi người.

Với công nghệ hiện đại phát triển như ngày nay, xây dựng một hệ thống chăm sóc sức khỏe linh hoạt và tiện lợi là điều đáng được quan tâm.

# MỤC LỤC

PHẦN MỘT: MỞ ĐẦU....1
1. Đặt vấn đề....1
2. Ý nghĩa lý thuyết, thực tiễn của đề tài....1
3. Mục tiêu của đề tài....2
4. Phạm vi của đề tài....2
5. Các phương pháp nghiên cứu....2
6. Kế hoạch thực hiện đề tài....2
PHẦN HAI: NỘI DUNG....4
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT....4
1.1. Giới thiệu NodeJs....4
1.1.1 Khái niệm....4
1.1.2 Ưu và nhược điểm của NodeJs....4
1.1.3 Node.JS hoạt động như thế nào?....4
1.2. Giới thiệu ReactJs....5
1.2.1 ReactJs là gì?....5
1.2.2 Ưu điểm của ReactJS....5
1.3. Giới thiệu ExpressJs....6
1.3.1 ExpressJs là gì?....6
1.3.2 ExpressJS được sử dụng để làm gì?....6
1.3.3 Những lợi ích của ExpressJS....6
1.3.4 Các tính năng của ExpressJS....7
1.4. Giới thiệu về MySQL....7
1.4.1 MySQL là gì?....7
1.4.2 Đặc điểm của MySQL....7
1.5. Giới thiệu về Xampp....8
1.5.1 XAMPP là gì?....8
1.5.2 Ưu điểm của XAMPP....8
1.5.3 Nhược điểm của XAMPP....9
1.5.4 XAMPP dùng để làm gì?....9
1.6. Giới thiệu Visual Studio Code....9
1.6.1 Visual Studio Code là gì?....9
1.6.2 Ưu điểm Visual Studio Code....10
1.6.3 Tại sao nên sử dụng Visual Studio Code?....10
1.7. Giới thiệu API....10

1.8. Phân tích hệ thống hướng đối tượng và UML ..... 13
1.8.1 Khái niệm phân tích hệ thống hướng đối tượng ..... 13
1.8.2 Khái niệm về UML (Unified Modeling Language) ..... 13
CHƯƠNG 2: XÂY DỰNG CHƯƠNG TRÌNH ..... 14
2.1 Đặc tả hệ thống ..... 14
2.2 Phân tích thiết kế hệ thống ..... 15
2.2.1 Sơ đồ hệ thống ..... 15
2.2.2 Sơ đồ Use Case ..... 16
2.2.2.1 Sơ đồ Use Case tổng quát ..... 16
2.2.2.2 Sơ đồ Use Case thành phần ..... 16
2.2.2.2.1 Use Case đăng nhập ..... 16
2.2.2.2.2 Use Case quản lý bác sĩ ..... 17
2.2.2.2.3 Use Case quản lý chuyên khoa ..... 17
2.2.2.2.4 Use Case quản lý tin tức ..... 18
2.2.2.2.5 Use Case quản lý đặt lịch khám ..... 19
2.2.2.2.6 Use Case quản lý người dùng ..... 19
2.2.2.2.7 Use Case cập nhật lịch trống ..... 20
2.2.2.2.8 Use Case xem thông tin bệnh nhân ..... 20
2.2.2.2.9 Use Case đặt lịch khám ..... 21
2.2.2.2.10 Use Case thanh toán lịch hẹn ..... 21
2.2.2.2.11 Use Case huỷ lịch hẹn khám ..... 22
2.2.3 Sơ đồ hoạt động ..... 22
2.2.3.1 Sơ đồ hoạt động đăng nhập ..... 22
2.2.3.2 Sơ đồ hoạt động quản lý dịch vụ ..... 23
2.2.3.3 Sơ đồ hoạt động quản lý chuyên khoa ..... 24
2.2.3.4 Sơ đồ hoạt động quản lý bác sĩ ..... 25
2.2.3.5 Sơ đồ hoạt động quản lý tin tức ..... 26
2.2.3.6 Sơ đồ hoạt động đặt lịch khám ..... 27
2.2.3.7 Sơ đồ hoạt động quản lý người dùng ..... 28
2.2.4 Sơ đồ thành phần ..... 29
2.3 Thiết kế cơ sở dữ liệu ..... 29
2.4 Một số giải thuật tiêu biểu ..... 37
2.4.1 Giải thuật tìm kiếm theo tên bác sĩ ..... 37
2.4.2 Giải thuật lập lịch theo thời gian bác sĩ ..... 37
2.5 Phát triển hệ thống ..... 38

2.5.1 Môi trường phát triển....38
2.5.2 Một số giao diện chính....38
2.5.2.1 Giao diện trang chủ....38
2.5.2.2 Giao diện chi tiết chuyên gia....39
2.5.2.3 Giao diện chi tiết dịch vụ....39
2.5.2.4 Giao diện tin tức....40
2.5.2.5 Giao diện đặt lịch khám....40
2.5.2.6 Giao diện trang chuyên khoa....41
2.5.2.7 Giao diện trang chuyên gia....41
2.5.2.8 Giao diện trang dịch vụ....42
2.5.2.9 Giao diện trang giới thiệu....42
2.5.2.10 Giao diện trang thông tin cá nhân....43
2.5.2.11 Giao diện thông tin liên hệ....43
PHẦN BA: PHẦN KẾT LUẬN....44
3.1 Kết quả đạt được....44
3.2 Hạn chế....44
3.3 Hướng phát triển....44
TÀI LIỆU THAM KHẢO....45
I. Giáo trình, bài giảng....45
II. Website....45

# DANH MỤC HÌNH

Hình 2. 1 Sơ đồ hệ thống .....15
Hình 2. 2 Sơ đồ Use Case tổng quát .....16
Hình 2. 3 Use Case đăng nhập .....17
Hình 2. 4 Use Case quản lý bác sĩ.....17
Hình 2. 5 Use Case quản lý chuyên khoa.....18
Hình 2. 6 Use Case quản lý tin tức.....18
Hình 2. 7 Use Case quản lý đặt lịch khám .....19
Hình 2. 8 Use Case quản lý người dùng.....19
Hình 2. 9 Use Case cập nhật lịch trống.....20
Hình 2. 10 Use Case xem thông tin bệnh nhân .....20
Hình 2. 11 Use Case đặt lịch khám.....21
Hình 2. 12 Use Case thanh toán lịch hẹn .....21
Hình 2. 13 Huỷ lịch hẹn.....22
Hình 2. 14 Sơ đồ hoạt động đăng nhập.....22
Hình 2. 15 Sơ đồ hoạt động quản lý dịch vụ.....23
Hình 2. 16 Sơ đồ hoạt động quản lý chuyên khoa .....24
Hình 2. 17 Sơ đồ hoạt động quản lý bác sĩ .....25
Hình 2. 18 Sơ đồ hoạt động quản lý tin tức .....26
Hình 2. 19 Sơ đồ hoạt động đặt lịch khám.....27
Hình 2. 20 Sơ đồ hoạt động quản lý người dùng .....28
Hình 2. 21 Sơ đồ thành phần.....29
Hình 2. 22 Sơ đồ lớp.....36
Hình 2. 23 Giao diện trang chủ.....38
Hình 2. 24 Giao diện chi tiết bác sĩ.....39
Hình 2. 25 Giao diện chi tiết dịch vụ .....39
Hình 2. 26 Giao diện tin tức.....40
Hình 2. 27 Giao diện đặt lịch khám .....40
Hình 2. 28 Giao diện trang chuyên khoa.....41
Hình 2. 29 Giao diện trang chuyên gia .....41
Hình 2. 30 Giao diện trang dịch vụ.....42
Hình 2. 31 Giao diện trang giới thiệu .....42
Hình 2. 32 Giao diện trang thông tin cá nhân .....43
Hình 2. 33 Giao diện thông tin liên hệ.....43

# DANH MỤC BẢNG

Bảng 2. 1 Bảng bác sĩ.....29
Bảng 2. 2 Bảng bài đăng.....30
Bảng 2. 3 Bảng chuyên khoa.....30
Bảng 2. 4 Bảng danh mục.....31
Bảng 2. 5 Bảng danh mục bài đăng.....31
Bảng 2. 6 Bảng dịch vụ.....32
Bảng 2. 7 Bảng dịch vụ bác sĩ.....32
Bảng 2. 8 Bảng giới thiệu.....32
Bảng 2. 9 Bảng lịch đặt lịch.....33
Bảng 2. 10 Lịch làm việc.....34
Bảng 2. 11 Bảng liên hệ.....34
Bảng 2. 12 Bảng người dùng.....35
Bảng 2. 13 Bảng nhãn.....35
Bảng 2. 14 Bảng nhãn bài đăng.....36

# PHẦN MỘT: MỞ ĐẦU

--- oOo ---

## 1. Đặt vấn đề

Việc quản lý và đặt lịch khám bệnh tại các phòng khám y tế ngày càng trở nên quan trọng và cần thiết trong bối cảnh người dân đang ngày càng tăng cường chăm sóc sức khỏe cá nhân và quá trình số hoá các hoạt động ngày càng nhiều. Tuy nhiên, nhiều phòng khám vẫn chưa tận dụng hết tiềm năng của công nghệ để tối ưu hóa quy trình đặt lịch, làm cho trải nghiệm của bệnh nhân và người quản lý phòng khám không thuận lợi, gây nhiều khó khăn cho bệnh nhân.

Nhiều phòng khám hiện nay đang gặp phải những thách thức trong việc quản lý lịch hẹn, từ việc xác định sự hiện diện của bác sĩ đến việc đảm bảo rằng bệnh nhân nhận được dịch vụ đúng đắn và đúng thời điểm. Điều này không chỉ ảnh hưởng đến hiệu suất hoạt động của phòng khám mà còn gây phiền toái cho bệnh nhân.

Chính vì vậy, đề tài nghiên cứu về xây dựng một hệ thống website đặt lịch khám tại phòng khám Hoàng Nam trở nên cấp thiết. Bằng cách này, chúng ta có thể giải quyết hiệu quả các vấn đề liên quan đến quản lý lịch hẹn, tăng cường tương tác giữa bác sĩ và bệnh nhân, đồng thời cải thiện trải nghiệm người dùng và nâng cao chất lượng dịch vụ y tế.

## 2. Ý nghĩa lý thuyết, thực tiễn của đề tài

### a) Ý nghĩa lý thuyết

Nghiên cứu về xây dựng website đặt lịch khám tại phòng khám Hoàng Nam mang lại nhiều giá trị lý thuyết quan trọng. Đầu tiên, đề tài đóng vai trò là một bước tiến trong lĩnh vực công nghệ y tế, áp dụng các nguyên tắc của quản lý thông tin và truy cập dữ liệu để cải thiện quy trình làm việc của phòng khám. Sự tích hợp của hệ thống thông tin này có thể giúp tối ưu hóa tài nguyên, giảm thiểu thời gian chờ đợi, và tăng cường khả năng phản hồi giữa bác sĩ và bệnh nhân.

Thứ hai, nghiên cứu này giúp thúc đẩy sự phát triển của các giải pháp công nghệ trong lĩnh vực y tế. Việc triển khai một hệ thống đặt lịch thông minh không chỉ giúp phòng khám Hoàng Nam mà còn có thể được chia sẻ và mở rộng cho các cơ sở y tế khác, tạo ra một cộng đồng chung hưởng lợi từ những tiến bộ trong quản lý lịch hẹn và tương tác bệnh nhân.

## 3. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng và triển khai thành công một hệ thống website đặt lịch khám cho phòng khám Hoàng Nam. Hệ thống này sẽ cung cấp một giao diện trực tuyến thuận tiện cho bệnh nhân để đặt lịch hẹn với các bác sĩ, đồng thời tối ưu hóa quy trình quản lý lịch trình và tương tác giữa bác sĩ và bệnh nhân.

## 4. Phạm vi của đề tài

Phạm vi triển khai hệ thống sẽ tập trung vào phòng khám Hoàng Nam, với khả năng mở rộng sang các chi nhánh khác trong tương lai.

## 5. Các phương pháp nghiên cứu

Để thực hiện đề tài, em đã sử dụng phương pháp nghiên cứu lý thuyết và phương pháp thực nghiệm để có thể hoàn thành được mục tiêu của đề tài đã đề ra. Cụ thể như sau:

Phương pháp nghiên cứu lý thuyết:

Nghiên cứu về hệ quản trị cơ sở dữ liệu MySQL

Nghiên cứu về xây dựng api bằng Node.Js Express

Nghiên cứu về xây dựng giao diện bằng React.

Nghiên cứu tìm hiểu về tình hình thực tế tại phòng khám, quá trình thực hiện đặt lịch, quy trình thực hiện.

Phân tích và thiết kế hệ thống đặt lịch.

Phương pháp nghiên cứu thực nghiệm: kết hợp với nghiên cứu tài liệu, quan sát, lấy ý kiến và thử nghiệm trên mô hình thực tế.

Tìm hiểu về ngôn ngữ lập trình JavaScript, React, Node.Js, Express và viết chương trình.

Quan sát các website và ứng dụng quản lý trên thị trường hiện có.

Chạy thử nghiệm và đánh giá kết quả đạt được.

## 6. Kế hoạch thực hiện đề tài

| STT | Thời gian | Nội dung thực hiện | Người thực hiện |
|---|---|---|---|
| 1 | 9/09/2023 – 15/09/2023 | Tìm tài liệu JavaScript, Node.Js, React, MySql, Express. | Nguyễn Hoàng Nam |
| 2 | 15/09/2023 – 30/09/2023 | Nghiên cứu nghiệp vụ, thiết kế | |

| | | | |
|---|---|---|---|
| | | và phân tích đề tài.<br>Xây dựng cơ sở dữ liệu. | |
| 3 | 1/10/2023 – 1/11/2023 | Thiết kế và phát triển hệ thống. | |
| 4 | 04/11/2023 – 14/11/2023 | Bước đầu hoàn chỉnh các chức năng hệ thống. | |
| 5 | 16/11/2023 – 20/11/2023 | Kiểm thử toàn hệ thống. | |
| 6 | 22/11/2023 – 1/12/2023 | Hiệu chỉnh các chức năng hệ thống. | |
| 7 | 2/12/2023 - 12/12/2023 | Tiếp tục hoàn thiện hệ thống.<br>Viết báo cáo. | |
| 8 | 13/12/2023 – 15/12/2023 | Hoàn thành báo cáo và sản phẩm. | |

# PHẦN HAI: NỘI DUNG

# CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

## 1.1. Giới thiệu NodeJs

### 1.1.1 Khái niệm

Node.js là một platform (nền tảng) độc lập được xây dựng trên môi trường Javascript Runtime - trình thông dịch ngôn ngữ Javascript cực nhanh trên trình duyệt Chrome. Dựa trên Node.js, chúng ta có thể xây dựng các ứng dụng mạng một cách nhanh chóng và dễ dàng mở rộng [1].

Node.js được thiết kế bởi Ryan Lienhart Dahl - một kỹ sư phần mềm người Mỹ vào năm 2009 và phát triển dưới sự bảo trợ của Joyent.

### 1.1.2 Ưu và nhược điểm của NodeJs

**a) Ưu điểm**

IO hướng sự kiện không đồng bộ, cho phép xử lý nhiều yêu cầu đồng thời.

Sử dụng JavaScript – một ngôn ngữ lập trình dễ học.

Chia sẻ cùng code ở cả phía client và server.

Node Package Manager và module Node đang ngày càng phát triển mạnh mẽ.

Cộng đồng hỗ trợ tích cực.

Cho phép stream các file có kích thước lớn

**a) Nhược điểm**

Không có khả năng mở rộng, vì vậy không thể tận dụng lợi thế mô hình đa lõi trong các phần cứng cấp server hiện nay.

Khó thao tác với cơ sử dữ liệu quan hệ.

Mỗi callback sẽ đi kèm với rất nhiều callback lồng nhau khác.

Cần có kiến thức tốt về JavaScript.

Không phù hợp với các tác vụ đòi hỏi nhiều CPU.

### 1.1.3 Node.JS hoạt động như thế nào?

Node.js hoạt động bằng cách thực thi mã JavaScript ở phía server, bên ngoài môi trường trình duyệt. Khi một ứng dụng Node.js được khởi động, nó sẽ tạo ra một vòng lặp sự kiện đơn luồng chờ các yêu cầu đến. Khi nhận được một yêu cầu, vòng lặp sự kiện sẽ xử lý yêu cầu đó một cách không đồng bộ mà không chặn luồng chính [1].

Mô hình I/O không chặn, không đồng bộ này là một trong những tính năng chính của Node.js. Nó cho phép một quy trình Node.js duy nhất xử lý đồng thời một số lượng lớn yêu cầu mà không bị cản trở bởi các hoạt động I/O chẳng hạn như I/O file hoặc I/O mạng. Điều này làm cho Node.js trở nên lý tưởng để xây dựng các ứng dụng mạng hiệu suất cao, có thể mở rộng.

Node.js sử dụng các mô-đun để tổ chức và tái sử dụng mã. Mô-đun là một khối mã độc lập xác định một hoặc nhiều hàm hoặc biến. Node.js bao gồm một hệ thống mô-đun tích hợp giúp dễ dàng nhập và sử dụng các mô-đun trong mã của bạn.

Mã Node.js có thể được thực thi bằng giao diện dòng lệnh (CLI) của Node.js, cho phép bạn chạy tệp JavaScript dưới dạng ứng dụng Node.js. Node.js cũng bao gồm một trình debugger tích hợp cho phép bạn xem qua mã của mình và debug trong thời gian thực [1].

## 1.2. Giới thiệu ReactJs

### 1.2.1 ReactJs là gì?

ReactJS là một opensource được phát triển bởi Facebook, ra mắt vào năm 2013, bản thân nó là một thư viện Javascript được dùng để để xây dựng các tương tác với các thành phần trên website. Một trong những điểm nổi bật nhất của ReactJS đó là việc render dữ liệu không chỉ thực hiện được trên tầng Server mà còn ở dưới Client nữa [5].

### 1.2.2 Ưu điểm của ReactJS

Ngoài việc hỗ trợ xây dựng giao diện nhanh, hạn chế lỗi trong quá trình code, cải thiện performance website thì những tính năng đặc biệt dưới đây:

Phù hợp với đa dạng thể loại website: ReactJS khiến cho việc khởi tạo website dễ dàng hơn bởi vì bạn không cần phải code nhiều như khi tạo trang web thuần chỉ dùng JavaScript, HTML và nó đã cung cấp cho bạn đủ loại "đồ chơi" để bạn có thể dùng cho nhiều trường hợp.

Tái sử dụng các Component: Nếu bạn xây dựng các Component đủ tốt, đủ flexible để có thể thoả các "yêu cầu" của nhiều dự án khác nhau, bạn chỉ tốn thời gian xây dựng ban đầu và sử dụng lại hầu như toàn bộ ở các dự án sau. Không chỉ riêng mỗi ReactJS mà các framework hiện nay cũng đều cho phép chúng ta thực hiện điều đó, ví dụ Flutter chẳng hạn.

Có thể sử dụng cho cả Mobile application: Hầu hết chúng ta đều biết rằng ReactJS được sử dụng cho việc lập trình website, nhưng thực chất nó được sinh ra không chỉ làm mỗi đều đó. Nếu bạn cần phát triển thêm ứng dụng Mobile, thì hãy sử dụng thêm React Native – một framework khác được phát triển cũng chính Facebook, bạn có thể

dễ dàng “chia sẻ” các Component hoặc sử dụng lại các Business Logic trong ứng dụng.

Thân thiện với SEO: SEO là một phần không thể thiếu để đưa thông tin website của bạn lên top đầu tìm kiếm của Google. Bản chất ReactJS là một thư viện JavaScript, Google Search Engine hiện nay đã crawl và index được code JavaScript, tuy nhiên bạn cũng cần thêm một vài thư viện khác để hỗ trợ điều này nhé!

Debug dễ dàng: Facebook đã phát hành 1 Chrome extension dùng trong việc debug trong quá trình phát triển ứng dụng. Điều đó giúp tăng tốc quá trình release sản phẩm cung như quá trình coding của bạn.

Công cụ phát triển web hot nhất hiện nay: Nếu bạn nhìn vào số liệu thống kê từ Google Trend ở Việt Nam ở hình bên dưới, dạo lướt qua các trang tuyển dụng hàng đầu ở Việt Nam như Topdev, Itviec, v.v bạn sẽ thấy số lượng tuyển dụng cho vị trí React Developer là cực kỳ lớn cùng với mức lương vô cùng hấp dẫn và độ phổ biến hiện tại của ReactJS trên thị trường Việt Nam là như thế nào [5].

## 1.3. Giới thiệu ExpressJs

### 1.3.1 ExpressJs là gì?

ExpressJS là một framework ứng dụng web có mã nguồn mở và miễn phí được xây dựng trên nền tảng Node.js. ExpressJS được sử dụng để thiết kế và phát triển các ứng dụng web một cách nhanh chóng. Để hiểu ExpressJS, người dùng chỉ cần phải biết JavaScript, do đó nên việc xây dựng các ứng dụng web và API trở nên đơn giản hơn đối với các lập trình viên và nhà phát triển đã thành thạo JavaScript trước đó [2].

### 1.3.2 ExpressJS được sử dụng để làm gì?

ExpressJS sẽ giúp tổ chức kiến trúc back-end. Các nhà phát triển web thường sử dụng ExpressJS để triển khai kiến trúc MVC, điều này cho phép họ viết một codebase back-end bảo trì tương đối dễ dàng. Bởi vì ExpressJS hoạt động trên back-end, nên bạn có thể coi công nghệ này như một “bộ não đằng sau một trang web”.

### 1.3.3 Những lợi ích của ExpressJS

Giúp cho việc phát triển back-end dễ dàng hơn nhiều khi sử dụng ExpressJS

Mã JavaScript được diễn giải thông qua Google V8 JavaScript Engine của Node.js. Do đó, mã sẽ được thực hiện một cách nhanh chóng và dễ dàng.

ExpressJS rất đơn giản để tùy chỉnh và sử dụng theo nhu cầu.

Cung cấp một module phần mềm trung gian linh hoạt và rất hữu ích để thực hiện các tác vụ bổ sung theo phản hồi và yêu cầu.

Hỗ trợ phát triển ứng dụng theo mô hình MVC, đây là mô hình phổ biến cho việc lập trình web hiện nay.

### 1.3.4 Các tính năng của ExpressJS

Các tính năng nổi bật mà ExpressJs mang lại như là:

Phát triển máy chủ nhanh hơn: ExpressJS cung cấp nhiều tính năng phổ biến của Node.js dưới dạng hàm có thể dễ dàng sử dụng ở bất kỳ đâu trong chương trình. Điều này sẽ giúp rút ngắn thời gian để viết code [2].

Phần mềm trung gian: Phần mềm trung gian là một phần trong chương trình cho phép truy cập vào cơ sở dữ liệu, xem xét yêu cầu của khách hàng và các phần mềm trung gian khác. Tính năng này chịu trách nhiệm chính cho việc tổ chức chức năng khác nhau của ExpressJS.

Định tuyến: ExpressJS cung cấp một cơ chế định tuyến nâng cao giúp duy trì trạng thái của trang web.

Khuôn mẫu: ExpressJS cung cấp các công cụ tạo khuôn mẫu cho phép các nhà phát triển tạo nội dung động trên các trang web bằng việc xây dựng các mẫu HTML ở phía máy chủ.

Gỡ lỗi: Gỡ lỗi là yếu tố quan trọng để phát triển các ứng dụng web. ExpressJS giúp gỡ lỗi dễ dàng hơn bằng cách cung cấp một cơ chế có khả năng xác định chính xác phần ứng dụng web có lỗi.

## 1.4. Giới thiệu về MySQL

### 1.4.1 MySQL là gì?

MySQL là hệ quản trị cơ sở dữ liệu tự do nguồn mở phổ biến nhất thế giới và được các nhà phát triển rất ưa chuộng trong quá trình phát triển ứng dụng. Vì MySQL là hệ quản trị cơ sở dữ liệu tốc độ cao, ổn định và dễ sử dụng, có tính khả chuyển, hoạt động trên nhiều hệ điều hành cung cấp một hệ thống lớn các hàm tiện ích rất mạnh. Với tốc độ và tính bảo mật cao, MySQL rất thích hợp cho các ứng dụng có truy cập CSDL trên internet. MySQL là một trong những ví dụ rất cơ bản về Hệ Quản trị Cơ sở dữ liệu quan hệ sử dụng Ngôn ngữ truy vấn có cấu trúc (SQL) [6].

MySQL được sử dụng cho việc bổ trợ NodeJs, PHP, Perl, và nhiều ngôn ngữ khác, làm nơi lưu trữ những thông tin trên các trang web viết bằng NodeJs, PHP hay Perl,...

### 1.4.2 Đặc điểm của MySQL

MySQL có nhiều đặc điểm như là:

Hệ quản trị cơ sở dữ liệu MySQL chính là một trong những phần mềm quản trị CSDL dạng server based, hệ gần giống với SQL server of Microsoft.

MySQL là phần mềm quản lý dữ liệu thông qua CSDL. Và mỗi một CSDL đều có bảng quan hệ chứa dữ liệu riêng biệt.

MySQL có cơ chế quản lý sử dụng riêng giúp cho mỗi người sử dụng đều có thể quản lý cùng lúc một hay nhiều CSDL khác nhau. Và mỗi người dùng đều có 1 username và password để truy nhập và truy xuất đến CSDL. Khi truy vấn đến CSDL của MySQL, bạn phải cung cấp tài khoản và mật khẩu có quyền sử dụng cơ sở dữ liệu đó [6].

MySQL tương thích tốt với môi trường PHP, giúp hệ thống hoạt động mạnh mẽ. Bạn có thể viết hoặc chỉnh sửa code MySQL trên các IDE dành cho PHP

## 1.5. Giới thiệu về Xampp

### 1.5.1 XAMPP là gì?

XAMPP là một phần mềm cho phép giả lập môi trường server hosting ngay trên máy tính của bạn, cho phép bạn chạy demo website mà không cần phải mua hosting hay VPS. Chính vì vậy, XAMPP hay được phục vụ cho hoạt động học tập giảng dạy thực hành và phát triển web.

XAMPP được viết tắt của X + Apache + MySQL + PHP + Perl vì nó được tích hợp sẵn Apache, MySQL, PHP, FTP server, Mail Server. Còn X thể hiện cho sự đa nền tảng của XAMPP vì nó có thể dùng được cho 4 hệ điều hành khác nhau: Windows, MacOS, Linux và Solaris [4].

### 1.5.2 Ưu điểm của XAMPP

Bên cạnh những tính năng mà XAMPP đem lại cho chúng ta, XAMPP còn có những ưu điểm để trở thành công cụ học tập và làm việc tốt như là:

Có thể chạy được linh hoạt được trên mọi hệ điều hành: Cross-platform, Windows, MacOS, Linux.

Có cấu hình đơn giản nhưng mang đến nhiều chức năng hữu ích như: Lập Server giả định, lập Mail Server giả định và hỗ trợ SSL trên localhost.

Tích hợp được nhiều tính năng với các thành phần quan trọng như: Apache, PHP, MySql. Vì thế, người dùng không cần cài đặt từng phần trên riêng lẻ mà chỉ cần cài XAMPP là có 1 web server hoàn chỉnh.

Tạo mã nguồn mở: Giao diện quản lý dễ dàng và tiện lợi giúp người dùng luôn chủ động được trong chế độ khởi động lại hay bật/ tắt đối với máy chủ theo thời gian phù hợp nhất [4].

### 1.5.3 Nhược điểm của XAMPP

Bên cạnh những ưu điểm mà XAMPP mang lại, thì XAMPP có một số ưu điểm không tốt như là:

Do cấu hình khá đơn giản cho nên Xampp hoàn toàn không nhận được sự hỗ trợ về cấu hình Module, đồng thời cũng không có cả MySQL.

Dung lượng tương đối nặng (141Mb).

Không có nhiều phiên bản cho từng thành phần của server như PHP, Apache mà phải cài đặt riêng.

### 1.5.4 XAMPP dùng để làm gì?

Sau khi chúng ta đã tìm hiểu về XAMPP là gì, ưu điểm và nhược điểm của XAMPP, vậy chúng ta cần biết XAMPP dùng để làm gì trong học tập và làm việc. XAMPP được dùng để xây dựng và phát triển website theo ngôn ngữ PHP. Ngoài ra, XAMPP còn được sử dụng để phát triển, nghiên cứu website thông qua localhost của máy tính cá nhân, biến máy tính cá nhân thành máy chủ, dùng chính ổ cứng của máy tính để làm nơi lưu trữ cho máy chủ trang web [4].

Tuy nhiên, đối với các trang web kinh doanh, các trang web cần vận hành liên tục thì việc sử dụng XAMPP để tạo lập không khả thi. Do localhost sử dụng máy tính để làm máy chủ, nên việc duy trì máy chủ sẽ rất khó khăn và không đảm bảo tốc độ lâu dài, thay vào đó các chủ trang web sẽ thuê server hosting. Chung quy lại XAMPP dùng chủ yếu để học tập, nâng cấp và thử nghiệm web.

## 1.6. Giới thiệu Visual Studio Code

### 1.6.1 Visual Studio Code là gì?

Visual Studio Code chính là ứng dụng cho phép biên tập, soạn thảo các đoạn code để hỗ trợ trong quá trình thực hiện xây dựng, thiết kế website một cách nhanh chóng. Visual Studio Code hay còn được viết tắt là VS Code. Trình soạn thảo này vận hành mượt mà trên các nền tảng như Windows, macOS, Linux. Hơn thế nữa, VS Code còn cho khả năng tương thích với những thiết bị máy tính có cấu hình tầm trung vẫn có thể sử dụng dễ dàng.

### 1.6.2 Ưu điểm Visual Studio Code

Visual Studio Code cũng luôn có những cải tiến và tạo ra đa dạng các tiện ích đi kèm từ đó giúp cho các lập trình viên sử dụng dễ dàng hơn. Trong đó có thể kể đến những ưu điểm sau:

Đa dạng ngôn ngữ lập trình giúp người dùng thỏa sức sáng tạo và sử dụng như HTML, CSS, JavaScript, C++.

Ngôn ngữ, giao diện tối giản, thân thiện, giúp các lập trình viên dễ dàng định hình nội dung.

Các tiện ích mở rộng rất đa dạng và phong phú.

Tích hợp các tính năng quan trọng như tính năng bảo mật (Git), khả năng tăng tốc xử lý vòng lặp.

Đơn giản hóa việc tìm quản lý hết tất cả các Code có trên hệ thống.

### 1.6.3 Tại sao nên sử dụng Visual Studio Code?

Không phải ngẫu nhiên mà Visual Studio Code được các lập trình viên ưa chuộng sử dụng. Visual Studio Code mang rất nhiều ưu điểm vượt trội so với bất kỳ IDE nào khác:

Hỗ trợ đa nền tảng: Linux, Mac, Windows.

Hỗ trợ đa ngôn ngữ: C/C++, C#, JavaScript, JSON, Visual Basic, HTML, CSS...

Ít dung lượng

Tính năng mạnh mẽ

Intellisense chuyên nghiệp

Giao diện thân thiện

Kiến trúc mạnh mẽ và người dùng có thể khai thác mở rộng

Số lượng người sử dụng lớn tạo nên cộng đồng hỗ trợ rộng rãi

## 1.7. Giới thiệu API

### 1.7.1 API là gì?

API là các phương thức, giao thức kết nối với các thư viện và ứng dụng khác. Nó là viết tắt của Application Programming Interface – giao diện lập trình ứng dụng. API cung cấp khả năng truy xuất đến một tập các hàm hay dùng. Và từ đó có thể trao đổi dữ liệu giữa các ứng dụng [3].

### 1.7.2 API thường ứng dụng vào đâu?

Web API: là hệ thống API được sử dụng trong các hệ thống website. Hầu hết các website đều ứng dụng đến Web API cho phép bạn kết nối, lấy dữ liệu hoặc cập nhật cơ sở dữ liệu. Ví dụ: Bạn thiết kế chức năng login thông Google, Facebook, Twitter, Github… Điều này có nghĩa là bạn đang gọi đến API của. Hoặc như các ứng dụng di động đều lấy dữ liệu thông qua API.

API trên hệ điều hành: Windows hay Linux có rất nhiều API, họ cung cấp các tài liệu API là đặc tả các hàm, phương thức cũng như các giao thức kết nối. Nó giúp lập trình viên có thể tạo ra các phần mềm ứng dụng có thể tương tác trực tiếp với hệ điều hành.

API của thư viện phần mềm hay framework: API mô tả và quy định các hành động mong muốn mà các thư viện cung cấp. Một API có thể có nhiều cách triển khai khác nhau và nó cũng giúp cho một chương trình viết bằng ngôn ngữ này có thể sử dụng thư viện được viết bằng ngôn ngữ khác. Ví dụ bạn có thể dùng Php để yêu cầu một thư viện tạo file PDF được viết bằng C++ [3].

### 1.7.3 Web API là gì?

Web API là một phương thức dùng để cho phép các ứng dụng khác nhau có thể giao tiếp, trao đổi dữ liệu qua lại. Dữ liệu được Web API trả lại thường ở dạng JSON hoặc XML thông qua giao thức HTTP hoặc HTTPS.

### 1.7.4 Điểm nổi bật của Web API

Web API hỗ trợ restful đầy đủ các phương thức: Get/Post/put/delete dữ liệu. Nó giúp bạn xây dựng các HTTP service một cách rất đơn giản và nhanh chóng. Nó cũng có khả năng hỗ trợ đầy đủ các thành phần HTTP: URI, request/response headers, caching, versioning, content format [3].

Tự động hóa sản phẩm: Với web API, chúng ta sẽ tự động hóa quản lý công việc, cập nhật luồng công việc, giúp tăng năng suất và tạo hiệu quả công việc cao hơn.

Khả năng tích hợp linh động: API cho phép lấy nội dung từ bất kỳ website hoặc ứng dụng nào một cách dễ dàng nếu được cho phép, tăng trải nghiệm người dùng. API hoạt động như một chiếc cổng, cho phép các công ty chia sẻ thông tin được chọn nhưng vẫn tránh được những yêu cầu không mong muốn.

Cập nhật thông tin thời gian thực: API có chức năng thay đổi và cập nhật thay đổi theo thời gian thực. Với công nghệ này, dữ liệu sẽ được truyền đi tốt hơn, thông tin chính xác hơn, dịch vụ cung cấp linh hoạt hơn.

Có tiêu chuẩn chung dễ sử dụng: Bất kỳ người dùng, công ty nào sử dụng cũng có thể điều chỉnh nội dung, dịch vụ mà họ sử dụng. Hỗ trợ đầy đủ các thành phần MVC như: routing, controller, action result, filter, model binder, IoC container, dependency injection, unit test.

### 1.7.5 Web API hoạt động như thế nào?

Đầu tiên là xây dựng URL API để bên thứ ba có thể gửi request dữ liệu đến máy chủ cung cấp nội dung, dịch vụ thông qua giao thức HTTP hoặc HTTPS.

Tại web server cung cấp nội dung, các ứng dụng nguồn sẽ thực hiện kiểm tra xác thực nếu có và tìm đến tài nguyên thích hợp để tạo nội dung trả về kết quả.

Server trả về kết quả theo định dạng JSON hoặc XML thông qua giao thức HTTP/HTTPS.

Tại nơi yêu cầu ban đầu là ứng dụng web hoặc ứng dụng di động, dữ liệu JSON/XML sẽ được parse để lấy data. Sau khi có được data thì thực hiện tiếp các hoạt động như lưu dữ liệu xuống Cơ sở dữ liệu, hiển thị dữ liệu…

### 1.7.6 Ưu và nhược điểm của Web API

**a) Ưu điểm**

Web API được sử dụng hầu hết trên các ứng dụng desktop, ứng dụng mobile và ứng dụng website.

Linh hoạt với các định dạng dữ liệu khi trả về client: Json, XML hay định dạng khác.

Nhanh chóng xây dựng HTTP service: URI, request/response headers, caching, versioning, content formats và có thể host trong ứng dụng hoặc trên IIS.

Mã nguồn mở, hỗ trợ chức năng RESTful đầy đủ, sử dụng bởi bất kì client nào hỗ trợ XML, Json.

Hỗ trợ đầy đủ các thành phần MVC như: routing, controller, action result, filter, model binder, IoC container, dependency injection, unit test.

Giao tiếp hai chiều được xác nhận trong các giao dịch, đảm bảo độ tin cậy cao.

**b) Nhược điểm**

Do web API còn khá mới nên chưa thể đánh giá nhiều về nhược điểm của mô hình nay. Tuy nhiên, có hai nhược điểm dễ dàng nhận thấy:

Web API chưa hoàn toàn phải là RESTful service, mới chỉ hỗ trợ mặc định GET, POST

Để sử dụng hiệu quả cần có kiến thức chuyên sâu, có kinh nghiệm backend tốt

Tốn thời gian và chi phí cho việc phát triển, nâng cấp và vận hành

Có thể gặp vấn đề về bảo mật khi hệ thống bị tấn công nếu không giới hạn điều kiện kỹ.

### 1.7.7 RESTful API là gì?

RESTful API là một tiêu chuẩn dùng trong việc thiết kế API cho các ứng dụng web (thiết kế Web services) để tiện cho việc quản lý các resource. Nó chú trọng vào tài nguyên hệ thống (tệp văn bản, ảnh, âm thanh, video, hoặc dữ liệu động…), bao gồm các trạng thái tài nguyên được định dạng và được truyền tải qua HTTP [3].

### 1.7.8 Cách hoạt động của RESTful API

REST hoạt động chủ yếu dựa vào giao thức HTTP. Các hoạt động cơ bản sẽ sử dụng những phương thức HTTP riêng như là: GET, POST, PUT, DELETE.

## 1.8. Phân tích hệ thống hướng đối tượng và UML

### 1.8.1 Khái niệm phân tích hệ thống hướng đối tượng

Phân tích hệ thống hướng đối tượng (Object-Oriented System Analysis - OOSA) là quá trình phân tích hệ thống sử dụng các nguyên tắc và khái niệm của lập trình hướng đối tượng (OOP). Phân tích hệ thống hướng đối tượng tập trung vào việc hiểu và mô hình hóa các thực thể, thuộc tính, mối quan hệ, và hành vi trong hệ thống một cách hướng đối tượng [8].

### 1.8.2 Khái niệm về UML (Unified Modeling Language)

UML (Unified Modeling Language) là ngôn ngữ dành cho việc đặc tả, hình dung, xây dựng và làm tài liệu của các hệ thống phần mềm. UML được sử dụng để thiết kế hệ thống, bao gồm những khái niệm như tiến trình nghiệp vụ và các chức năng của hệ thống. Cụ thể, nó hữu dụng cho những ngôn ngữ khai báo, giản đồ cơ sở dữ liệu, thành phần phần mềm có khả năng tái sử dụng. UML được phát triển bởi Grady Booch, Ivar Jabcobsen và James Rumbaugh vào năm 1990, nó nhanh chóng trở thành một trong những ngôn ngữ chuẩn để xây dựng hệ thống phần mềm hướng đối tượng (Object-Oriented) [7]. Hiện tại UML đã có phiên bản 2.5. Có các loại sơ đồ UML chủ yếu sau:

Sơ đồ lớp (Class Diagram)

Sơ đồ thành phần (Component Diagram)

Sơ đồ hoạt động (Activity Diagram)

# CHƯƠNG 2: XÂY DỰNG CHƯƠNG TRÌNH

## 2.1 Đặc tả hệ thống

Trong quá trình phát triển phần mềm, giai đoạn triển khai và bảo trì đóng vai trò quan trọng trong việc đảm bảo sự liên tục và hiệu quả của hệ thống. Khác với giai đoạn phát triển chính, giai đoạn triển khai và bảo trì tập trung chủ yếu vào trải nghiệm của người dùng và sự ổn định của hệ thống khi đã đi vào hoạt động. Trong tổng thể, giai đoạn triển khai và bảo trì không chỉ là quá trình kết thúc một dự án mà còn là cơ hội để tối ưu hóa trải nghiệm người dùng và duy trì sự linh hoạt để đáp ứng thách thức ngày càng đa dạng trong môi trường hoạt động thực tế.

Đối với trong lĩnh vực quản lý y tế tại phòng khám, việc đặt lịch khám bệnh của bệnh nhân tại phòng khám đó thực hiện thủ công thì rất là mất thời gian và công sức, cho nên ta cần một hệ thống để tối ưu hoá việc đặt lịch thủ công giúp người bệnh có thể đặt lịch tại nhà mà không cần đến cơ sở khám mất thời gian và công sức. Vì vậy hệ thống đặt lịch khám bệnh tại phòng khám ra đời giúp cải thiện quá trình đặt lịch khám bệnh rườm rà phức tạp, giúp trải nghiệm người bệnh tại phòng khám trở nên dễ dàng, tiện dụng. Nghiệp vụ của hệ thống được mô tả như sau:

Đối với người dùng là bệnh nhân: Đầu tiên khi vào hệ thống, người dùng có thể xem đầy đủ các thông tin của phòng khám, thông tin bác sĩ, dịch vụ của phòng khám. Khi người dùng có nhu cầu đặt lịch khám bệnh, người dùng có thể gọi trực tiếp để đặt lịch với phòng khám hoặc là đặt lịch qua hệ thống. Khi người dùng đặt lịch qua hệ thống, người dùng cần có tài khoản để đăng nhập vào hệ thống, nếu chưa có thì người dùng cần đăng kí tài khoản để có thể bắt đầu đặt lịch. Khi đã đăng nhập thành công, người dùng có thể bắt đầu đặt lịch khám bác sĩ. Tại đây, người dùng có thể chọn loại dịch vụ cần khám, chọn bác sĩ muốn khám và khung thời gian mà người dùng có thể khám được, sau đó điền đầy đủ thông tin để tiến hành đặt lịch. Khi đã hoàn tất người dùng bắt đầu đặt lịch thì người dùng cần thanh toán một khoản tiền nhỏ để có thể đặt lịch khám được với bác sĩ. Khi đã thanh toán thành công, người dùng và quản lý tài khoản sẽ thấy được lịch sử đặt lịch của mình, tại đây người dùng có thể xem các thông tin, xem thông tin bác sĩ, liên hệ với bác sĩ.

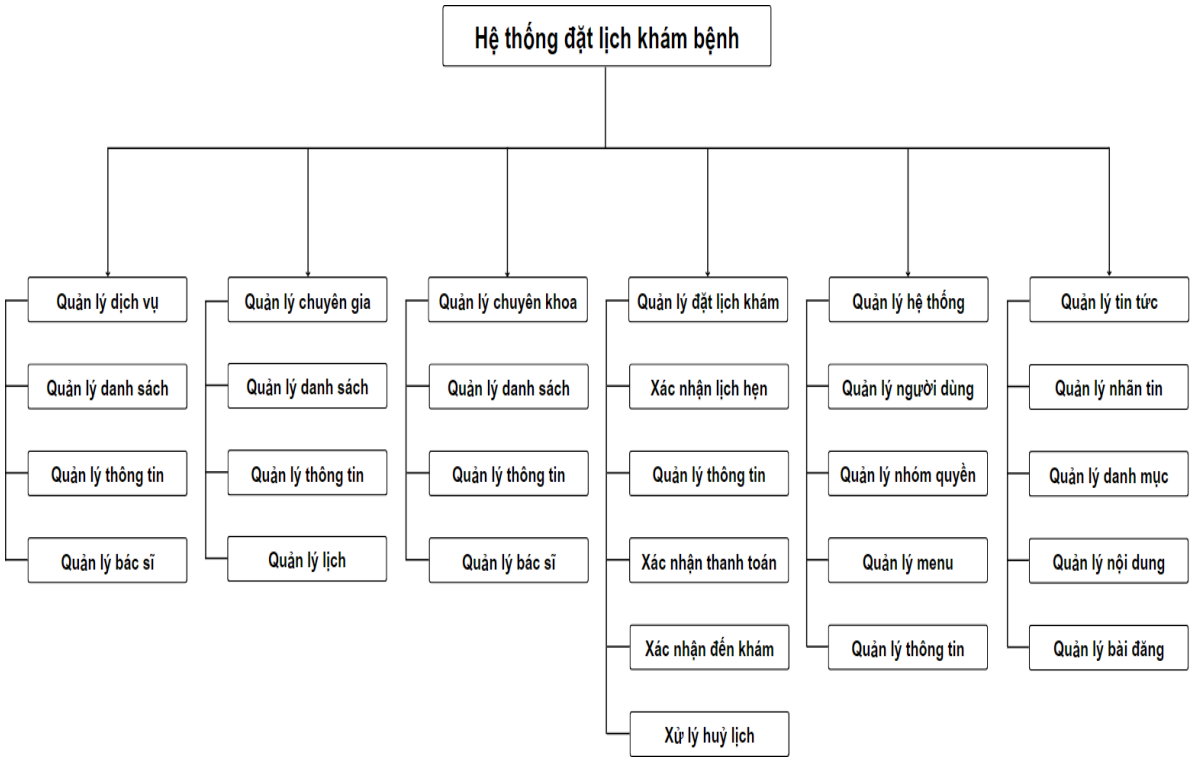
Đối với người dùng là bác sĩ: khi vào hệ thống người dùng cần đăng nhập vào hệ thống, mật khẩu và tài khoản sẽ do quản trị viên cao nhất cấp. Khi vào hệ thống, người dùng chọn menu quản lý lịch khám bệnh, tại đây người dùng có thể xem bệnh nhân đã đặt lịch của mình thành công, thông tin của bệnh nhân, ngày khám. Bên cạnh đó, người dùng có thể cập nhật lịch trống của mình theo tháng và các buổi trống trong ngày để bệnh nhân có thể đặt lịch theo lịch trống của bác sĩ.

Đối với người dùng là quản trị viên: người dùng này có quyền cao nhất trong hệ thống, người dùng có thể cấp tài khoản mật khẩu có 2 loại người dùng là khách hàng và bác sĩ. Bên cạnh đó, người dùng có thể thực hiện các chức năng quản lý thông tin bác sĩ, thêm sửa xoá thông tin bác sĩ, quản lý số lượng bệnh nhân đặt lịch, quản lý các loại dịch vụ tại phòng khám, có thể thêm sửa xoá các loại dịch vụ đó, người dùng có thể quản lý các chuyên khoa, các bác sĩ trong chuyên khoa, thêm sửa xoá các thông tin trong chuyên khoa. Cùng với các chức năng đó, người dùng này có quyền thay đổi các logo, các hình ảnh giới thiệu, nội dung phòng khám trên hệ thống. Nghiệp vụ quan trọng của người dùng này là khi người dùng là khách hàng thanh toán tiền bằng hình thức chuyển khoản thì người dùng là quản trị viên sẽ nhân được và cập nhật trạng thái cho người dùng là khách hàng biết là đã nhận được và hoá đơn đặt lịch đó đã được thanh toán thành công để người dùng là khách hàng có thể gặp bác sĩ để tư vấn.

## 2.2 Phân tích thiết kế hệ thống

### 2.2.1 Sơ đồ hệ thống

Hình dưới đây sẽ mô tả các các chức mà ba loại người dùng có thể thực hiện trên hệ thống như là các chức năng về đặt lịch, thanh toán, quản lý người dùng, quản lý dịch vụ, quản lý chuyên gia, quản lý chuyên khoa, quản lý hệ thống …




Hình 2. 1 Sơ đồ hệ thống

### 2.2.2 Sơ đồ Use Case

#### 2.2.2.1 Sơ đồ Use Case tổng quát




Hình 2. 2 Sơ đồ Use Case tổng quát

Mô tả các actor:

Khách hàng: là những người dùng trực tiếp các chức năng trên hệ thống như là đặt lịch khám, thanh toán...

Bác sĩ: là những người được cấp tài khoản để quản lý lịch làm việc và quản lý bệnh nhân đặt lịch của mình.

Quản trị viên: là người quản trị cao nhất của hệ thống, quản trị viên có quyền quản lý các thông tin của hệ thống như bác sĩ, loại dịch vụ, các chuyên khoa, cấp tài khoản… Quản trị quyền có quyền thêm sửa xoá các thông tin của hệ thống.

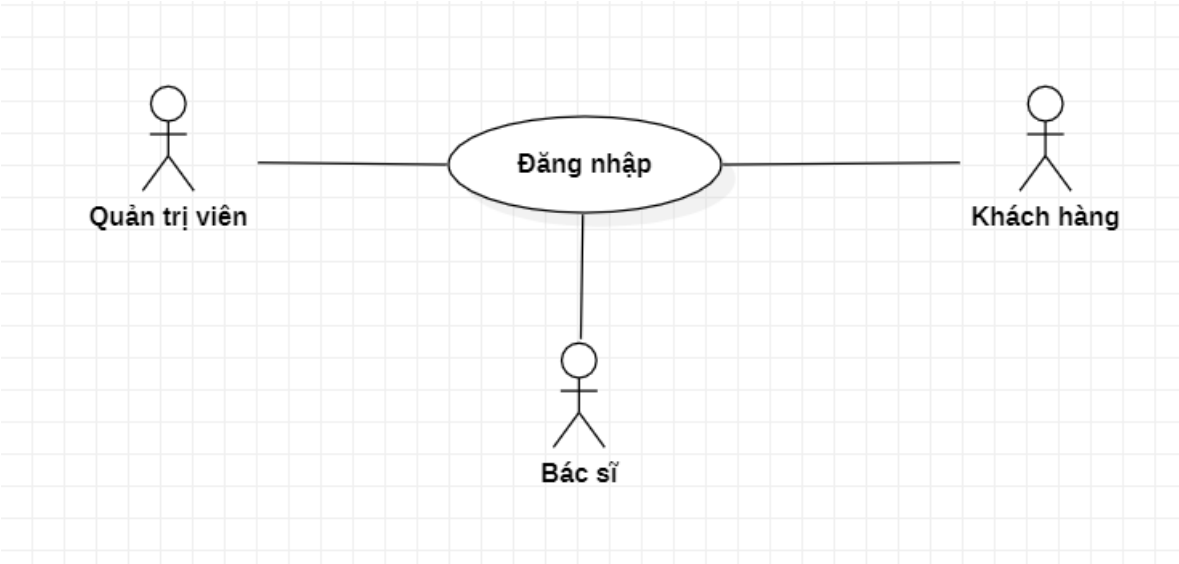
#### 2.2.2.2 Sơ đồ Use Case thành phần

##### 2.2.2.2.1 Use Case đăng nhập

Tác nhân: Quản trị viên, bác sĩ, khách hàng

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Quản trị viên / bác sĩ / khách hàng đăng nhập vào hệ thống bằng tài khoản và mật khẩu cá nhân. Tùy vào phân quyền của tài khoản, quản trị viên / bác sĩ / khách hàng có thể truy cập các chức năng tương ứng của hệ thống.



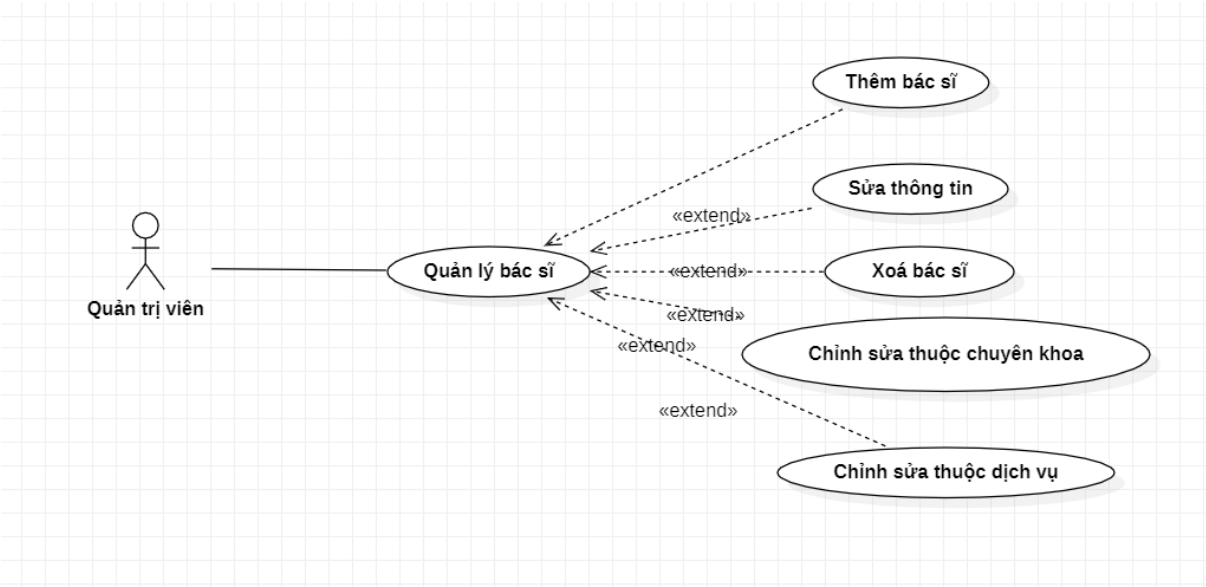

Hình 2. 3 Use Case đăng nhập

#### 2.2.2.2.2 Use Case quản lý bác sĩ

Tác nhân: Quản trị viên

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong quản lý bác sĩ, người dùng là quản trị viên có quyền thêm sửa xoá các thông tin của bác sĩ, bên cạnh đó có thể chỉnh sửa chuyên khoa của bác sĩ và chỉnh sửa dịch vụ mà bác sĩ đó đang đảm nhiệm trên hệ thống.



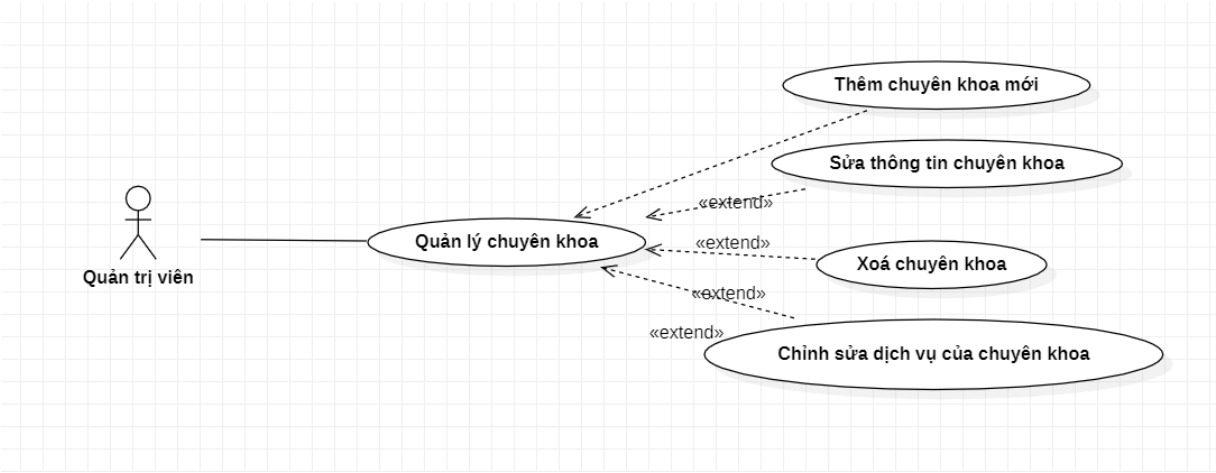

Hình 2. 4 Use Case quản lý bác sĩ

#### 2.2.2.2.3 Use Case quản lý chuyên khoa

Tác nhân: Quản trị viên

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong quản lý chuyên khoa, người dùng là quản trị viên có quyền thêm sửa xoá các thông tin của chuyên khoa đó, bên cạnh đó có thể chỉnh sửa hoặc thêm các dịch vụ của phòng khám cung cấp thuộc chuyên khoa.



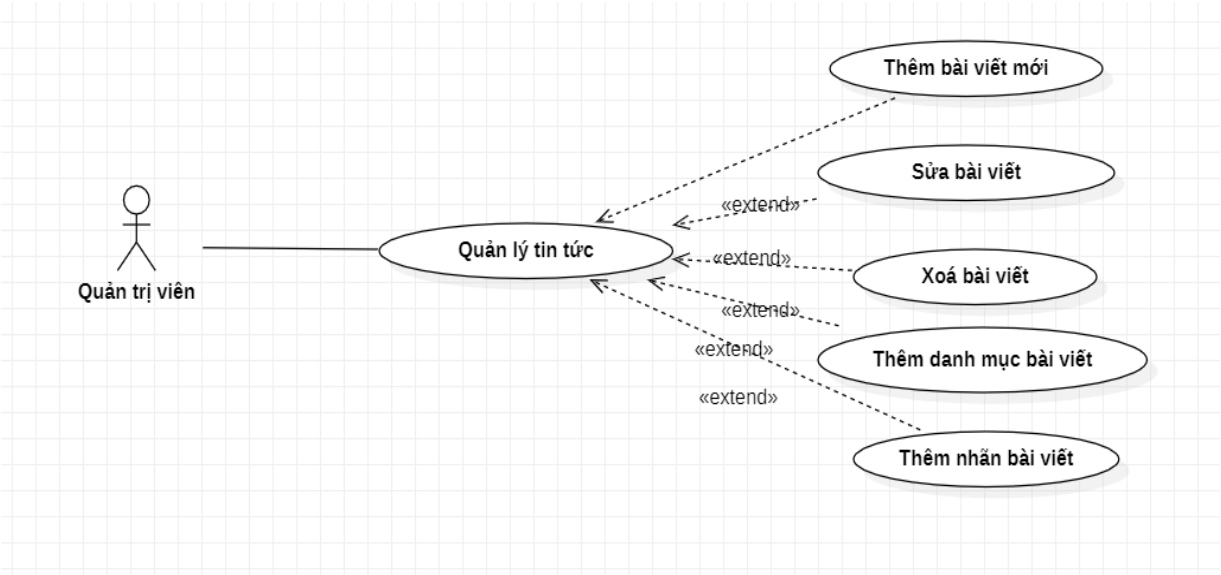

Hình 2. 5 Use Case quản lý chuyên khoa

#### 2.2.2.2.4 Use Case quản lý tin tức

Tác nhân: Quản trị viên

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong quản lý tin tức, người dùng là quản trị viên có quyền thêm sửa xoá các bài đăng của hệ thống, bên cạnh đó người dùng có thể thêm các danh mục, thêm các nhãn trong quản lý tin tức.



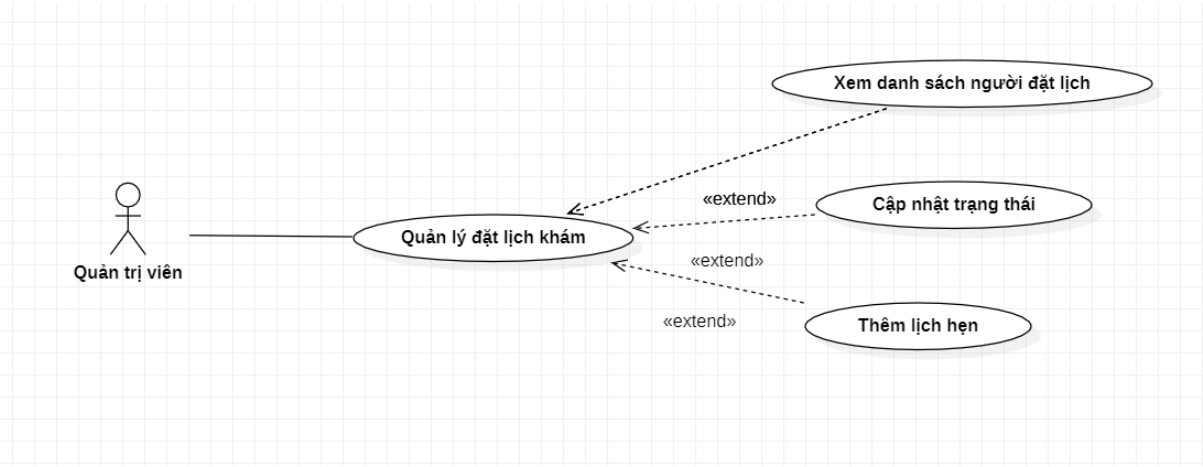

Hình 2. 6 Use Case quản lý tin tức

#### 2.2.2.2.5 Use Case quản lý đặt lịch khám

Tác nhân: Quản trị viên

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong quản lý đặt lịch khám, người dùng là quản trị viên có quyền xem danh sách của những khách hàng đã đặt lịch và xem thông tin đặt lịch trên hệ thống, bên cạnh đó người dùng thực hiện cập nhật trạng thái khi khách hàng đã hoàn thành từng quá trình như là thanh toán, đã gặp bác sĩ, khám thành công. Ngoài ra, quản trị viên còn có thể thêm một lịch hẹn mới vào hệ thống cho một khách hàng tại phòng khám.



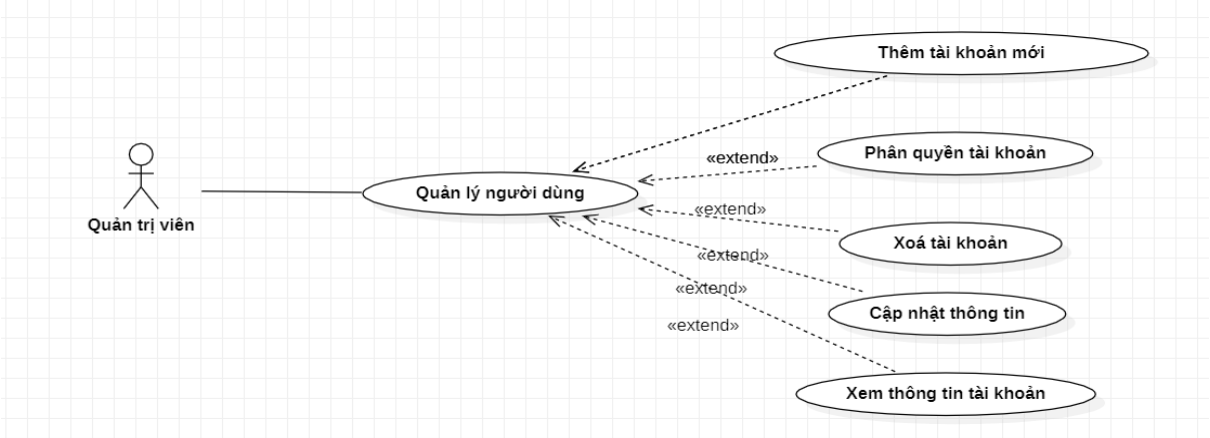

Hình 2. 7 Use Case quản lý đặt lịch khám

#### 2.2.2.2.6 Use Case quản lý người dùng

Tác nhân: Quản trị viên

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong quản lý người dùng, người dùng là quản trị viên có quyền thêm sửa xoá các tài khoản có trên hệ thống, bên cạnh đó người dùng có thể chỉnh sửa các quyền của tài khoản để phân loại tài khoản và xem thông tin chi tiết của tài khoản đó.



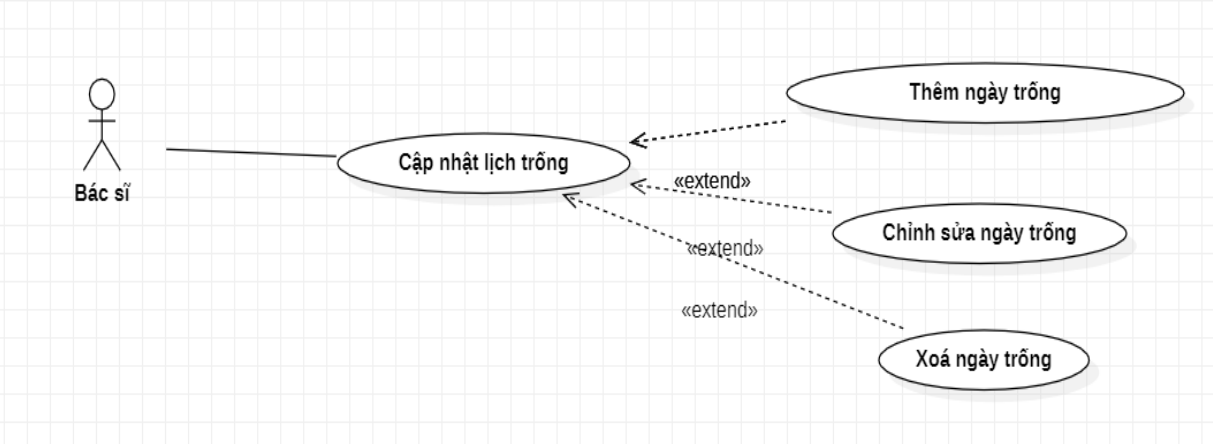

Hình 2. 8 Use Case quản lý người dùng

#### 2.2.2.2.7 Use Case cập nhật lịch trống

Tác nhân: Bác sĩ

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong cập nhật lịch trống, người dùng là bác sĩ có thể cập nhật lịch trống của bản thân lên hệ thống. Bên cạnh đó, bác sĩ chỉnh sửa và xoá ngày trống để phù hợp với lịch bản thân.



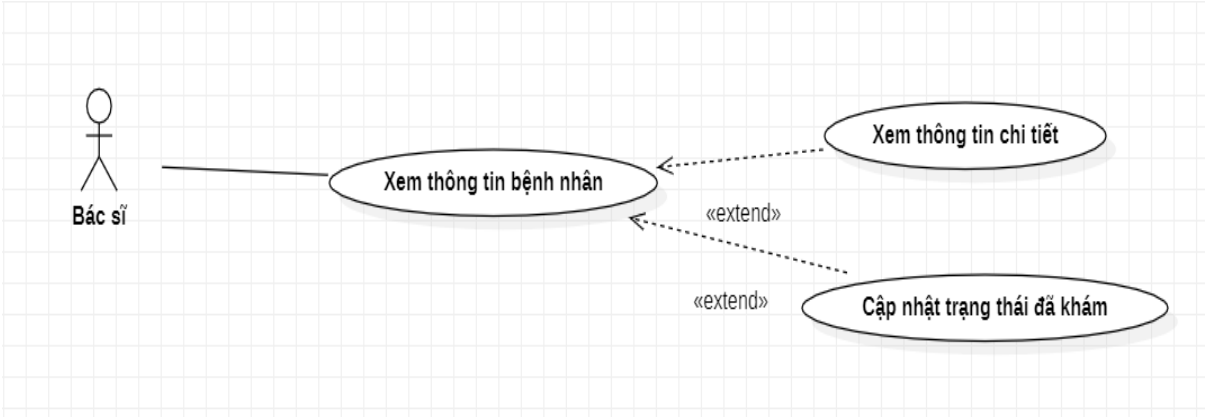

Hình 2. 9 Use Case cập nhật lịch trống

#### 2.2.2.2.8 Use Case xem thông tin bệnh nhân

Tác nhân: Bác sĩ

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong xem thông tin bệnh nhân, người dùng là bác sĩ có thể xem đầy đủ các thông tin chi tiết mà người dùng là khách hàng đã đặt lịch. Bên cạnh đó, người dùng là bác sĩ cần làm thêm một chức năng cập nhật trạng thái khi đã hoàn thành khám cho bệnh nhân.



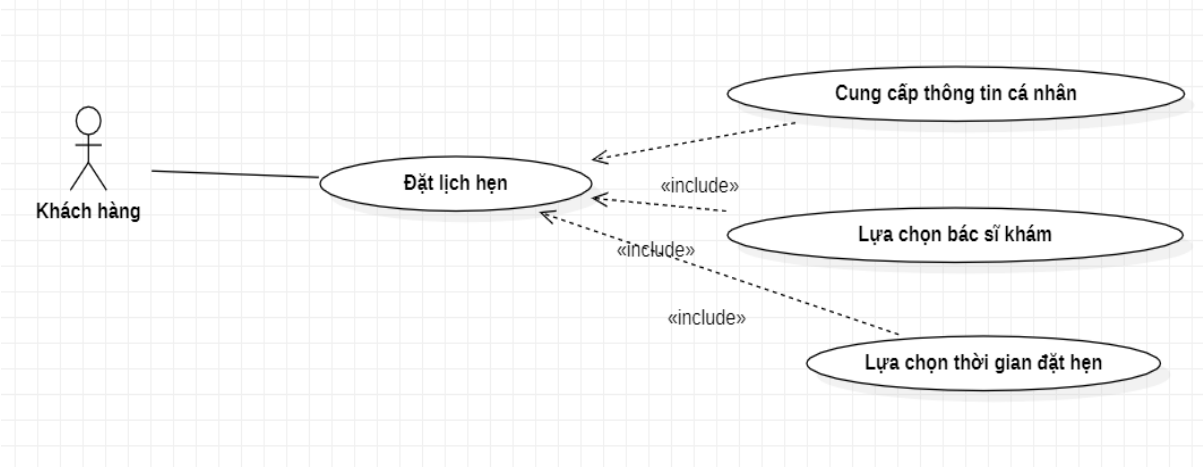

Hình 2. 10 Use Case xem thông tin bệnh nhân

#### 2.2.2.2.9 Use Case đặt lịch khám

Tác nhân: Khách hàng

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong đặt lịch khám, người dùng là khách hàng cần chọn bác sĩ khám và chọn mốc thời gian phù hợp với bản thân để đặt lịch khám.



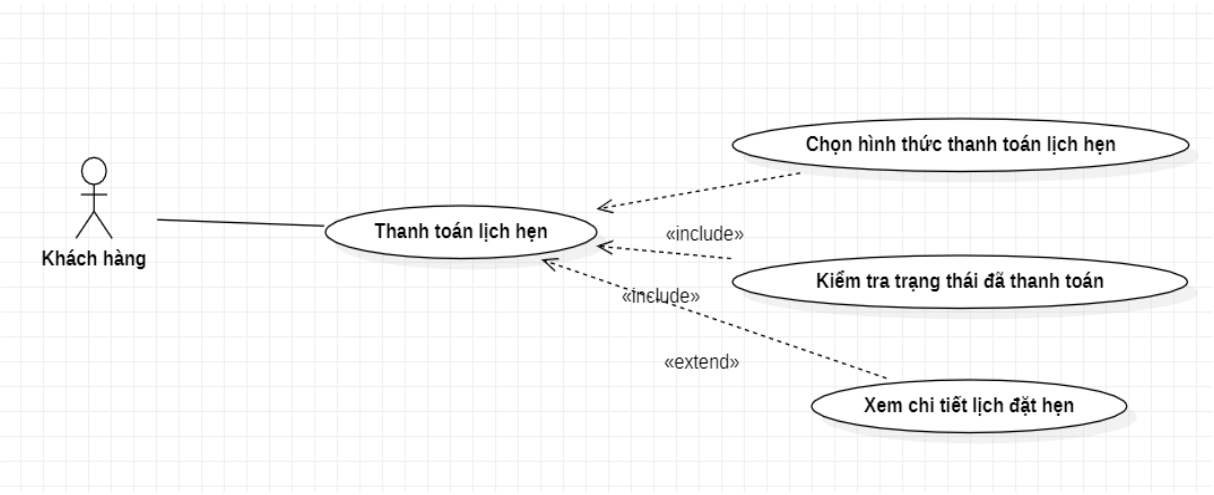

Hình 2. 11 Use Case đặt lịch khám

#### 2.2.2.2.10 Use Case thanh toán lịch hẹn

Tác nhân: Khách hàng

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong thanh toán lịch hẹn, sau khi đã đặt lịch hẹn, người dùng cần thanh toán để hoàn thành quá trình đặt lịch. Tại đây người dùng cần chọn hình thức trực tuyến trên hệ thống hay tại phòng khám và kiểm tra trạng thái của đơn đặt lịch. Bên cạnh đó người dùng có thể xem chi tiết đơn đặt lịch của mình.



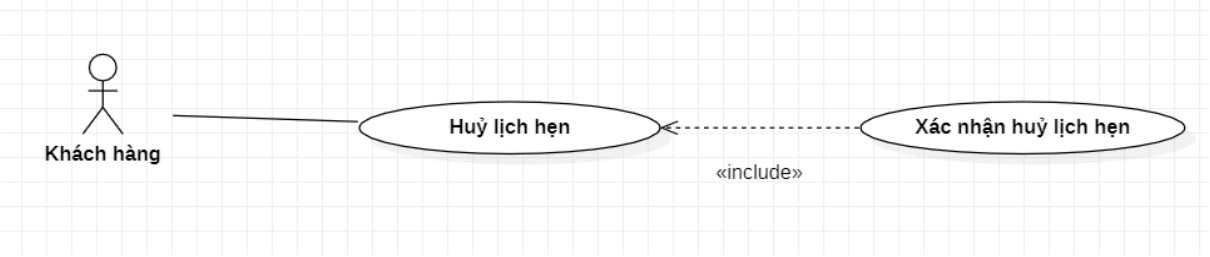

Hình 2. 12 Use Case thanh toán lịch hẹn

#### 2.2.2.2.11 Use Case huỷ lịch hẹn khám

Tác nhân: Khách hàng

Điều kiện: Có tài khoản, mật khẩu

Mô tả: Trong huỷ lịch hẹn, người dùng cần xác nhận huỷ lịch hẹn thì lịch đặt hẹn đó sẽ được huỷ theo yêu cầu của khách hàng.

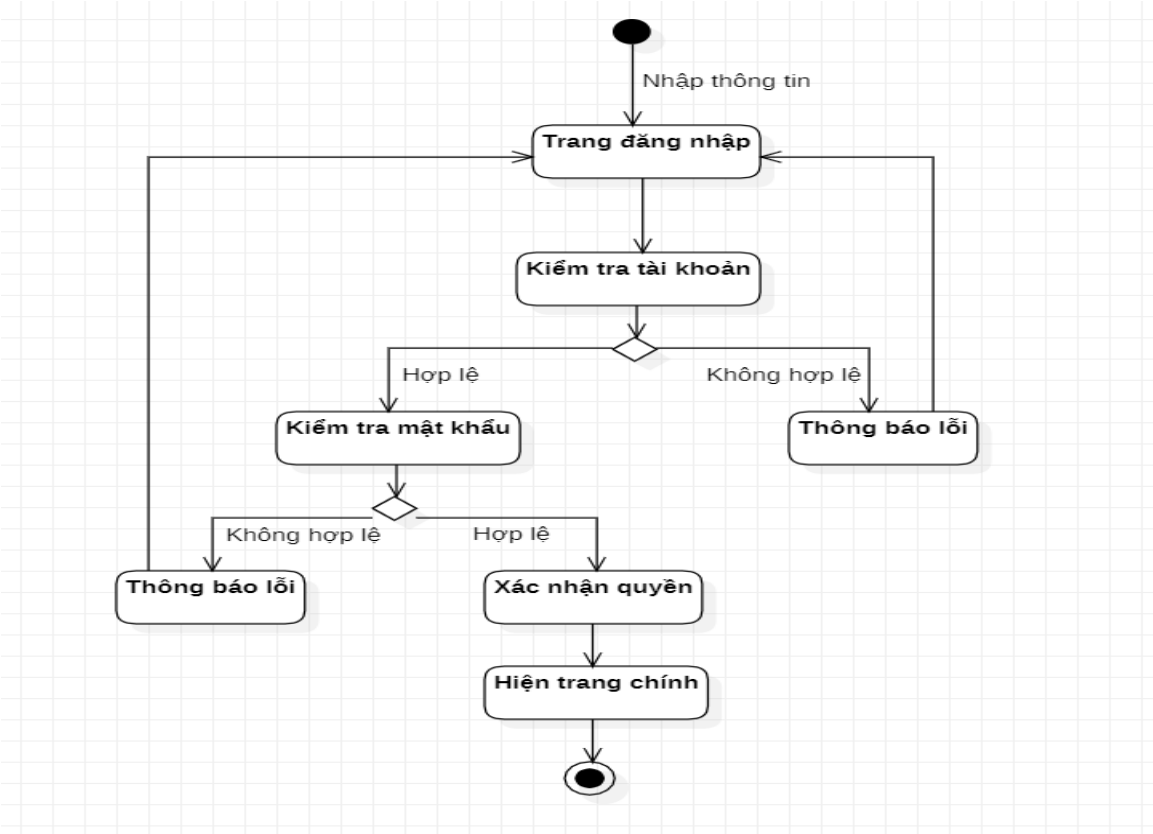



Hình 2. 13 Huỷ lịch hẹn

### 2.2.3 Sơ đồ hoạt động

#### 2.2.3.1 Sơ đồ hoạt động đăng nhập

Người dùng yêu cầu đăng nhập, người dùng nhập thông tin đăng nhập gồm tài khoản và mật khẩu tại trang đăng nhập. Sau đó, hệ thống kiểm tra tài khoản, nếu tài khoản hợp lệ thì "đăng nhập thành công", xác nhận quyền của tài khoản và hiện trang chủ hệ thống và kết thúc. Ngược lại, thông báo mật khẩu không chính xác và kết thúc. Còn nếu tài khoản không hợp lệ thì thông báo tài khoản không tồn tại và kết thúc.




Hình 2. 14 Sơ đồ hoạt động đăng nhập

### 2.2.3.2 Sơ đồ hoạt động quản lý dịch vụ

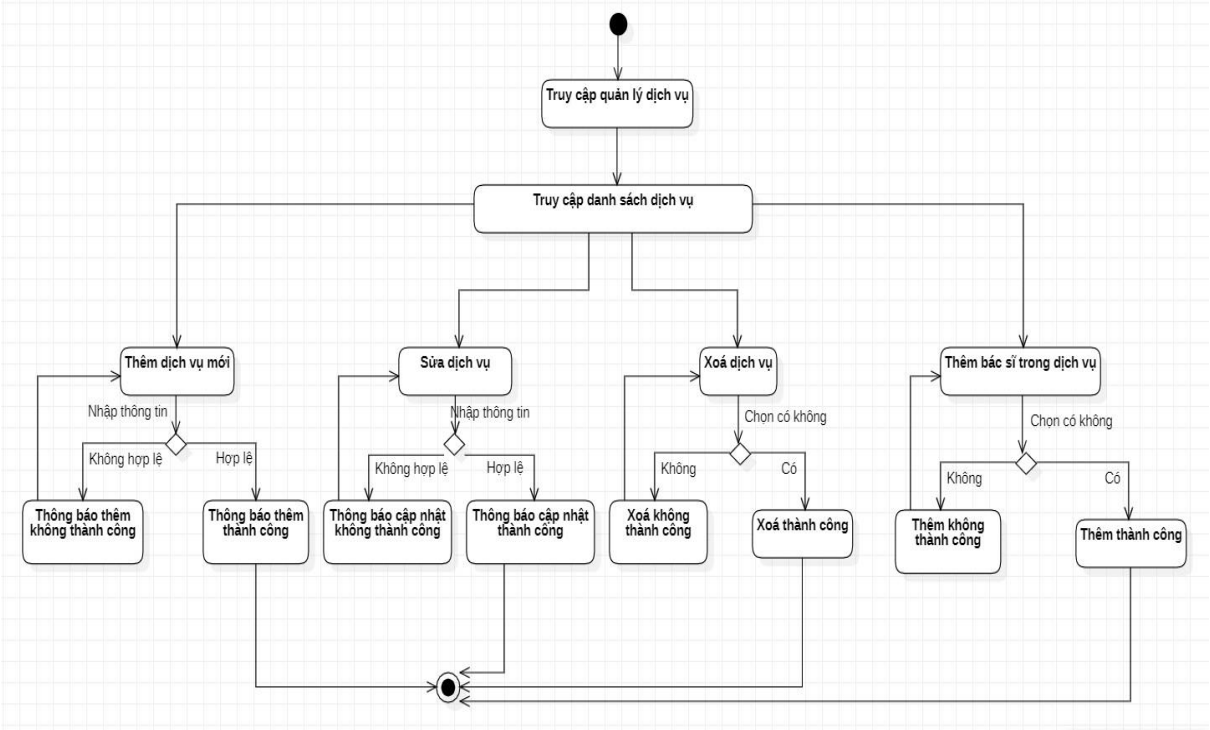
Trong quản lý dịch vụ, người dùng là quản trị viên có thể dùng các chức năng như thêm dịch vụ mới, sửa dịch vụ, xoá dịch vụ, thêm bác sĩ trong dịch vụ.

Trong hoạt động thêm dịch vụ mới: người dùng cần nhập các thông tin mà hệ thống yêu cầu để có thể thêm, sau khi đã nhập đầy đủ và thêm thì hệ thống sẽ hiển thị thông báo thêm thành công và kết thúc hoạt động thêm, ngược lại thì thông báo thêm không thành công và trở về hoạt động thêm dịch vụ.

Trong hoạt động sửa dịch vụ: người cần cần nhập những thông tin của dịch vụ muốn sửa, nếu hợp lệ và đầy đủ thì hệ thống sẽ thông báo cập nhật thành công và kết thúc hoạt động sửa dịch vụ, ngược lại thì thông báo cập nhật không thành công.

Trong hoạt động xoá dịch vụ: người dùng cần chọn dịch vụ muốn xoá, hệ thống sẽ hiển thị một thông báo lựa chọn, nếu người dùng chọn có thì hệ thông sẽ thông báo xoá thành công và kết thúc hoạt động này, ngược lại thì sẽ thông báo xoá không thành công, trở về hoạt động xoá như lúc đầu.

Trong hoạt động thêm bác sĩ trong dịch vụ: người dùng chọn bác sĩ để thêm, tại đây hệ thống sẽ thông báo lựa chọn có không cho người dùng, nếu người dùng chọn có hệ thống sẽ hiển thị thông báo thêm thành công và kết thúc hoạt động này, ngược lại thì thông báo thêm không thành công và quay trở lại ban đầu.




Hình 2. 15 Sơ đồ hoạt động quản lý dịch vụ

### 2.2.3.3 Sơ đồ hoạt động quản lý chuyên khoa

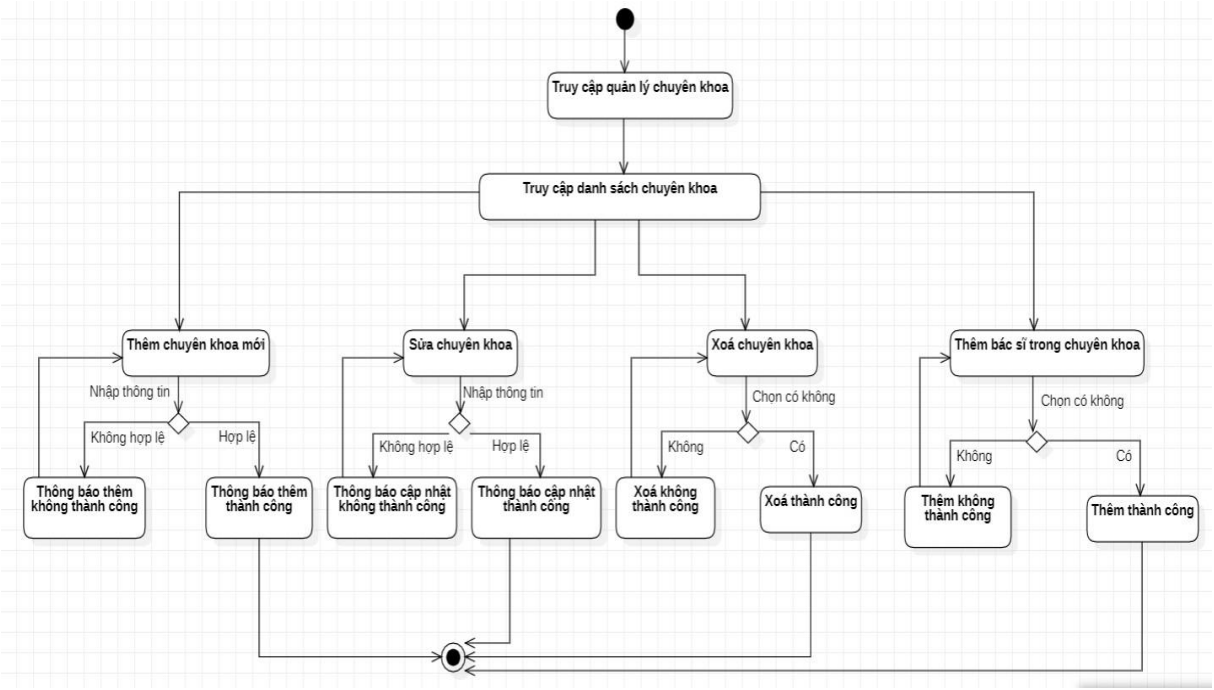
Trong quản lý chuyên khoa, người dùng là quản trị viên có thể dùng các chức năng như thêm chuyên khoa mới, sửa chuyên khoa, xoá chuyên khoa, thêm bác sĩ trong chuyên khoa.

Trong hoạt động thêm chuyên khoa mới: người dùng cần nhập các thông tin mà hệ thống yêu cầu để có thể thêm, sau khi đã nhập đầy đủ và thêm thì hệ thống sẽ hiển thị thông báo thêm thành công và kết thúc hoạt động thêm, ngược lại thì thông báo thêm không thành công và trở về hoạt động thêm chuyên khoa.

Trong hoạt động sửa chuyên khoa: người cần cần nhập những thông tin của chuyên khoa muốn sửa, nếu hợp lệ và đầy đủ thì hệ thống sẽ thông báo cập nhật thành công và kết thúc hoạt động sửa chuyên khoa, ngược lại thì thông báo cập nhật không thành công.

Trong hoạt động xoá chuyên khoa: người dùng cần chọn chuyên khoa muốn xoá, hệ thống sẽ hiển thị một thông báo lựa chọn, nếu người dùng chọn có thì hệ thông sẽ thông báo xoá thành công và kết thúc hoạt động này, ngược lại thì sẽ thông báo xoá không thành công, trở về hoạt động xoá như lúc đầu.

Trong hoạt động thêm bác sĩ trong chuyên khoa: người dùng chọn bác sĩ để thêm, tại đây hệ thống sẽ thông báo lựa chọn có không cho người dùng, nếu nguời dùng chọn có hệ thống sẽ hiển thị thông báo thêm thành công và kết thúc hoạt động này, ngược lại thì thông báo thêm không thành công và quay trở lại ban đầu.




Hình 2. 16 Sơ đồ hoạt động quản lý chuyên khoa

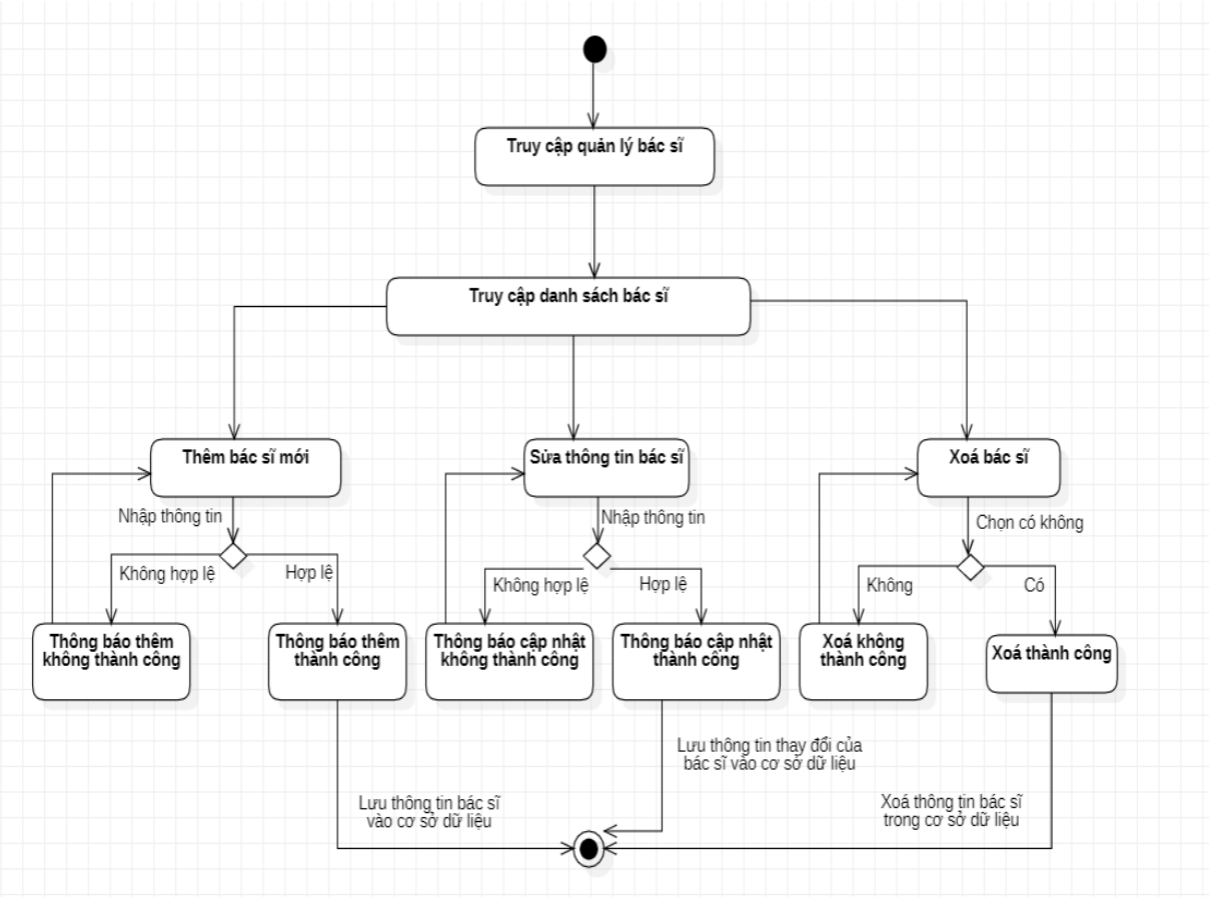
### 2.2.3.4 Sơ đồ hoạt động quản lý bác sĩ

Trong quản lý bác sĩ, người dùng là quản trị viên có thể dùng các chức năng như thêm bác sĩ mới, sửa thông tin bác sĩ, xoá bác sĩ trên hệ thống

Trong hoạt động thêm bác sĩ mới: người dùng cần nhập các thông tin mà hệ thống yêu cầu để có thể thêm, sau khi đã nhập đầy đủ và thêm thì hệ thống sẽ hiển thị thông báo thêm thành công và lưu thông tin bác sĩ vào cơ sở dữ liệu kết thúc hoạt động thêm, ngược lại thì thông báo thêm không thành công và trở về hoạt động thêm bác sĩ.

Trong hoạt động sửa bác sĩ: người cần cần nhập những thông tin của bác sĩ muốn sửa, nếu hợp lệ và đầy đủ thì hệ thống sẽ thông báo cập nhật thành công và lưu thay đổi thông tin bác sĩ vào cơ sở dữ liệu rồi kết thúc hoạt động sửa chuyên khoa, ngược lại thì thông báo cập nhật không thành công.

Trong hoạt động xoá bác sĩ: người dùng cần chọn bác sĩ muốn xoá, hệ thống sẽ hiển thị một thông báo lựa chọn, nếu người dùng chọn có thì hệ thông sẽ thông báo xoá thành công và xoá thông tin bác sĩ yêu cầu trong cơ sở dữ liệu rồi kết thúc hoạt động này, ngược lại thì sẽ thông báo xoá không thành công, trở về hoạt động xoá như lúc đầu.




Hình 2. 17 Sơ đồ hoạt động quản lý bác sĩ

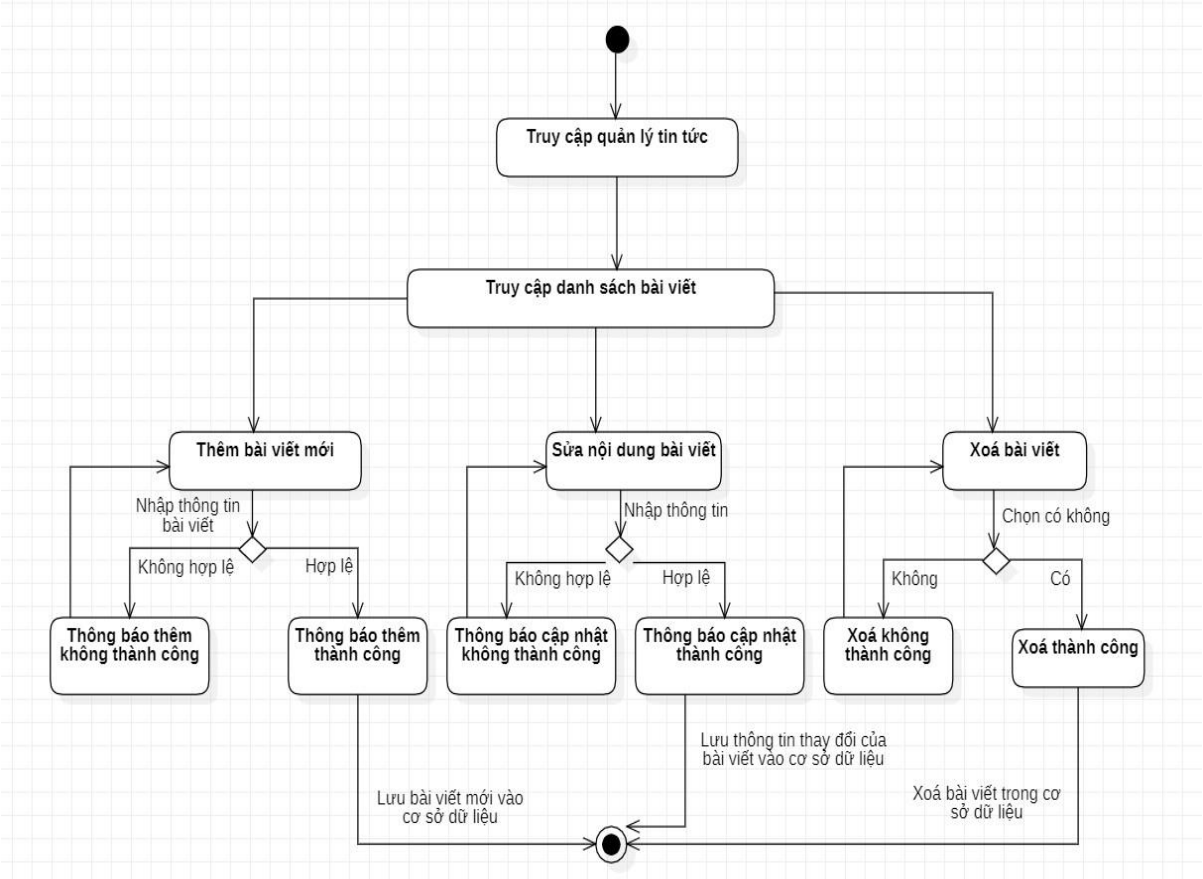
### 2.2.3.5 Sơ đồ hoạt động quản lý tin tức

Trong quản lý tin tức, người dùng là quản trị viên có thể dùng các chức năng như thêm bài viết mới, sửa thông tin bài viết, xoá bài viết trên hệ thống

Trong hoạt động thêm bài viết mới: người dùng cần nhập các thông tin mà hệ thống yêu cầu để có thể thêm, sau khi đã nhập đầy đủ và thêm thì hệ thống sẽ hiển thị thông báo thêm thành công và lưu thông tin bài viết vào cơ sở dữ liệu kết thúc hoạt động thêm, ngược lại thì thông báo thêm không thành công và trở về hoạt động thêm bác sĩ.

Trong hoạt động sửa bài viết: người cần cần nhập những thông tin của bài viết muốn sửa, nếu hợp lệ và đầy đủ thì hệ thống sẽ thông báo cập nhật thành công và lưu thông tin thay đổi bài viết vào cơ sở dữ liệu rồi kết thúc hoạt động sửa bài viết, ngược lại thì thông báo cập nhật không thành công.

Trong hoạt động xoá bài viết: người dùng cần chọn bài viết muốn xoá, hệ thống sẽ hiển thị một thông báo lựa chọn, nếu người dùng chọn có thì hệ thông sẽ thông báo xoá thành công và xoá thông tin bài viết yêu cầu trong cơ sở dữ liệu rồi kết thúc hoạt động này, ngược lại thì sẽ thông báo xoá không thành công, trở về hoạt động xoá như lúc đầu.



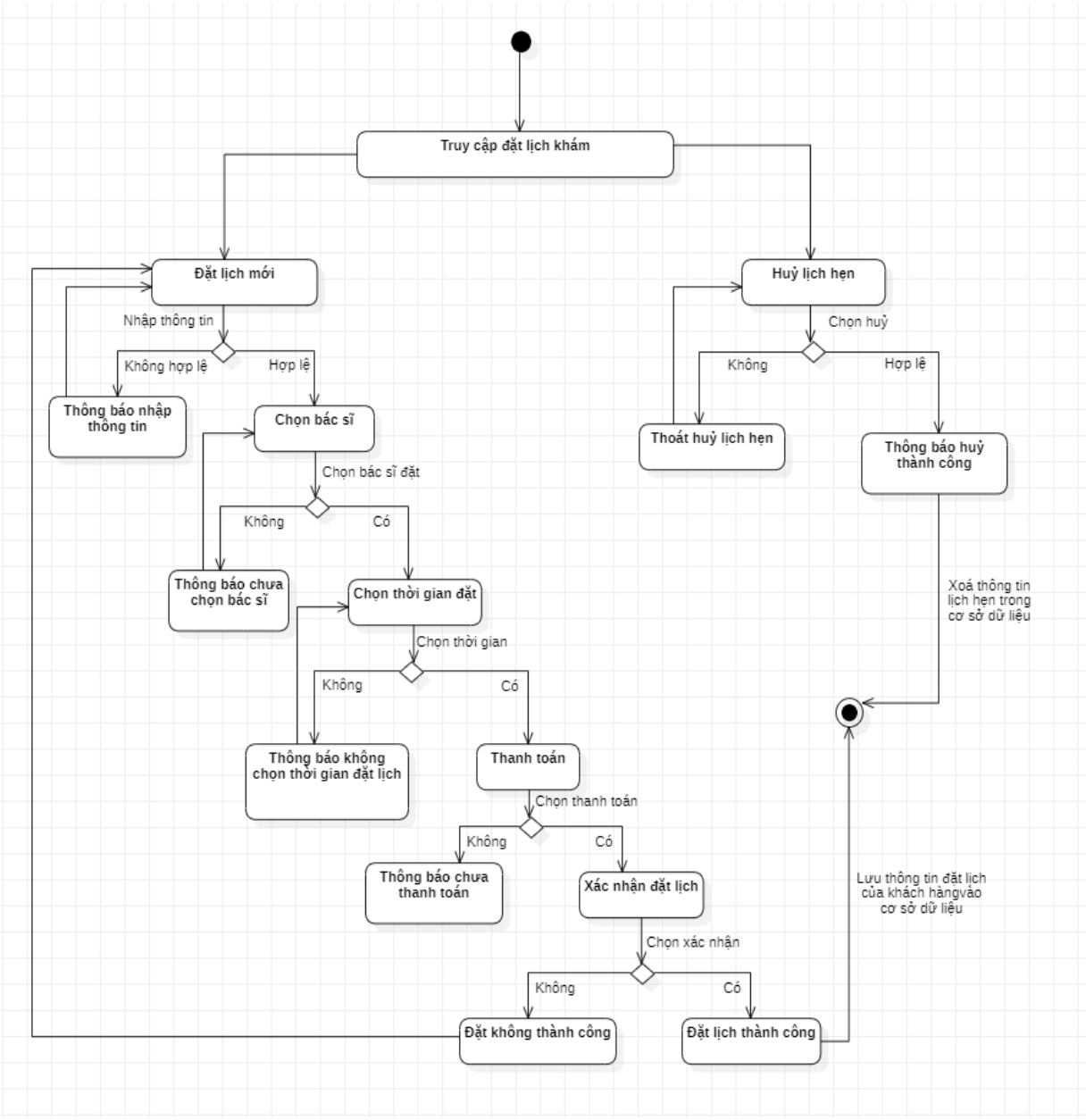

Hình 2. 18 Sơ đồ hoạt động quản lý tin tức

### 2.2.3.6 Sơ đồ hoạt động đặt lịch khám

Trong hoạt động đặt lịch khám, người dùng là khách hàng có thể đặt lịch mới hoặc là huỷ lịch hẹn.

Trong hoạt động đặt lịch mới: người dùng cần nhập các thông tin cá nhân để phòng khám có thể liên hệ sau đó chọn bác sĩ, chọn thời gian đặt lịch theo lịch trống của bác sĩ. Tiếp đến thì cần tiến hành thanh toán và xác nhận là đặt để hoạt động đặt lịch khám thành công.

Trong hoạt động huỷ lịch hẹn: người dùng có thể huỷ một lịch hẹn nào đó bất kì, khi chọn huỷ hệ thống sẽ thông báo cho người dùng lựa chọn. Nếu người dùng chọn không thì sẽ không huỷ lịch hẹn đó, ngược lại thì sẽ thông báo huỷ thành công và xoá lịch hẹn của khách hàng trong cơ sở dữ liệu.




Hình 2. 19 Sơ đồ hoạt động đặt lịch khám

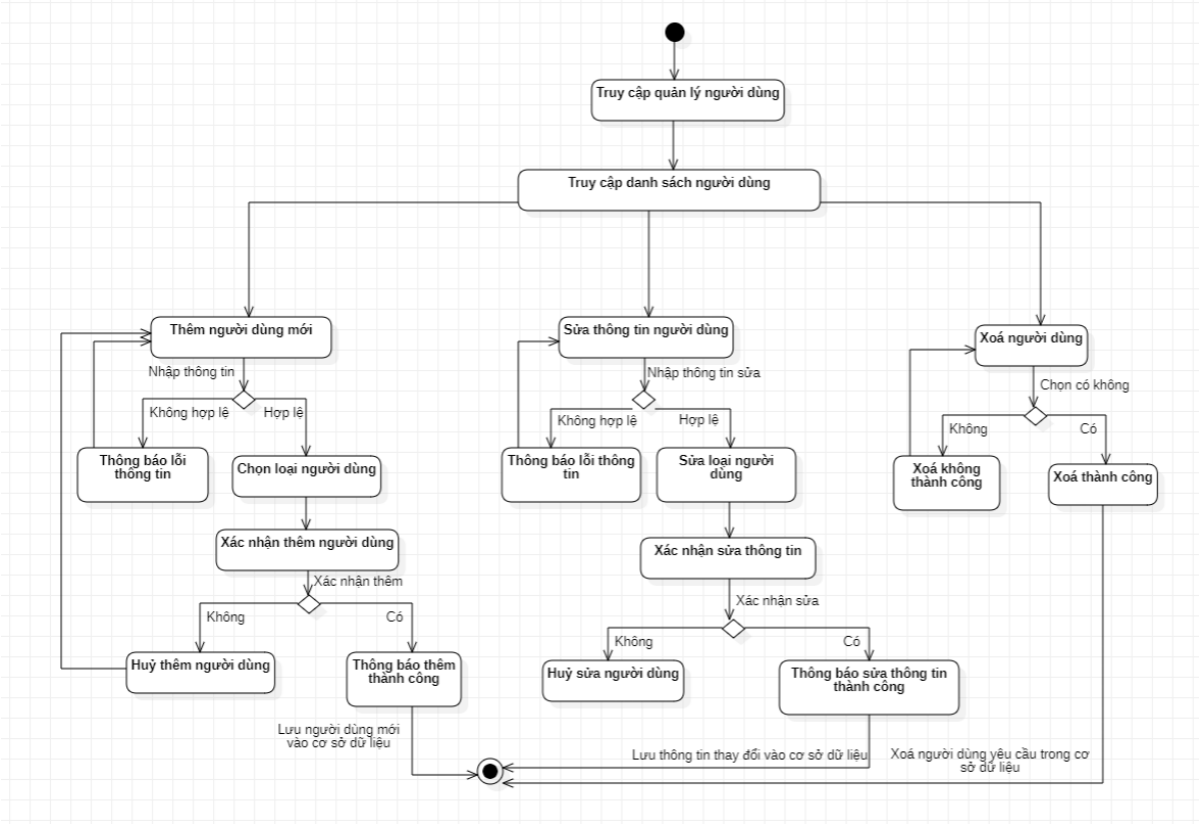
### 2.2.3.7 Sơ đồ hoạt động quản lý người dùng

Trong quản lý người dùng, người dùng là quản trị viên có thể dùng các chức năng như thêm người dùng mới, sửa thông tin người dùng, xoá người dùng trên hệ thống

Trong hoạt động thêm người dùng mới: người dùng cần nhập các thông tin mà hệ thống yêu cầu để có thể thêm, sau khi đã nhập đầy đủ, sau đó đến chọn loại người dùng, cuối cùng chọn xác nhận thêm thì hệ thống sẽ hiển thị thông báo thêm thành công và lưu thông tin người dùng vào cơ sở dữ liệu kết thúc hoạt động thêm, ngược lại thì thông báo thêm không thành công và trở về hoạt động thêm người dùng.

Trong hoạt động sửa người dùng: người cần cần nhập những thông tin của người dùng muốn sửa, sau đó cần chọn loại người dùng để thay đổi, cuối cùng thì chọn xác nhận. Nếu người dùng chọn xác nhận thì hệ thống sẽ thông báo cập nhật thành công và lưu thay đổi thông tin người dùng vào cơ sở dữ liệu rồi kết thúc hoạt động sửa người dùng, ngược lại thì thông báo cập nhật không thành công.

Trong hoạt động xoá người dùng: người dùng cần chọn người dùng muốn xoá, hệ thống sẽ hiển thị một thông báo lựa chọn, nếu người dùng chọn có thì hệ thông sẽ thông báo xoá thành công và xoá thông tin người dùng yêu cầu trong cơ sở dữ liệu rồi kết thúc hoạt động này, ngược lại thì sẽ thông báo xoá không thành công, trở về hoạt động xoá như lúc đầu




Hình 2. 20 Sơ đồ hoạt động quản lý người dùng

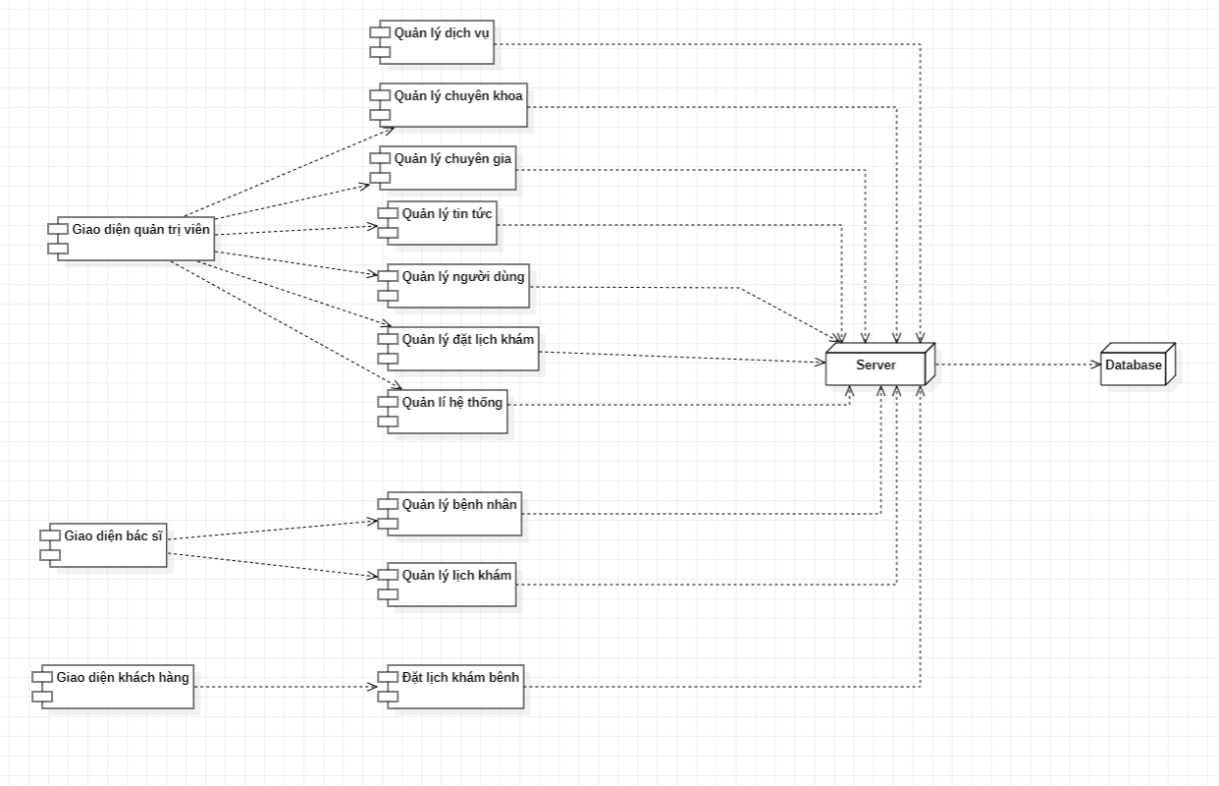
### 2.2.4 Sơ đồ thành phần

Sơ đồ thành phần sẽ khái quát giao diện về chức năng chính của hệ thống như là:

Giao diện người quản trị: quản lý chuyên khoa, quản lý chuyên gia, quản lý tin tức, quản lý người dùng, quản lý dịch vụ, quản lý hệ thống, quản lý đặt lịch khám.

Giao diện bác sĩ: quản lý bệnh nhân, quản lý lịch khám bệnh.

Giao diện khách hàng: đặt lịch khám bệnh.




Hình 2. 21 Sơ đồ thành phần

## 2.3 Thiết kế cơ sở dữ liệu

### a. Bảng tbl_bacsi

Bảng tbl_bacsi dùng để lưu các thông tin của các bác sĩ tại phòng khám như là tên, ảnh, giới thiệu, chuyên môn,…

Bảng 2. 1 Bảng bác sĩ

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaBacSi | int | Mã bác sĩ, Khoá chính |
| 2 | TenBacSi | varchar(500) | Tên bác sĩ |
| 3 | GioiThieuNgan | varchar(2000) | Giới thiệu ngắn |
| 4 | AnhBacSi | varchar(5000) | Ảnh bác sĩ |

| | | | |
|---|---|---|---|
| 5 | ThongTin | text | Thông tin |
| 6 | GioiThieu | text | Giới thiệu |
| 7 | ThanhVien | text | Thành viên |
| 8 | LinhVucChuyenMon | text | Lĩnh vực chuyên môn |
| 9 | GiaiThuong | text | Giải thưởng |
| 10 | CongTrinh | text | Công trình |
| 11 | QuaTrinhDaoTao | text | Quá trình đào tạo |
| 12 | KinhNghiem | text | Kinh nghiệm |
| 13 | DaXoa | bit(10) | Đã xoá |

**b. Bảng tbl_baidang**

Bảng tbl_baidang dùng để lưu các thông tin của bài viết trong hệ thống của phòng khám như là tiêu đề, mô tả ngắn, nội dung, ngày đăng,…

Bảng 2. 2 Bảng bài đăng

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaBaiDang | int | Mã bài đăng, Khoá chính |
| 2 | TieuDe | varchar(1000) | Tiêu đề |
| 3 | MoTaNgan | varchar(5000) | Mô tả ngắn |
| 4 | NoiDung | text | Nội dung |
| 5 | NgayDang | timestamp(6) | Ngày đăng |
| 6 | MaNguoiDung | int | Mã người dùng, Khoá ngoại |
| 7 | DaXoa | bit(10) | Đã xoá |

**c. Bảng tbl_chuyenkhoa**

Bảng tbl_chuyenkhoa dùng để lưu các thông tin của các chuyên khoa trong hệ thống của phòng khám như là tên chuyên khoa, giới thiệu, thông tin chi tiết,…

Bảng 2. 3 Bảng chuyên khoa

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaChuyenKhoa | int | Mã chuyên khoa, Khoá chính |

| | | | |
|---|---|---|---|
| 2 | TenChuyenKhoa | varchar(500) | Tên chuyên khoa |
| 3 | AnhBia | varchar(5000) | Ảnh bìa |
| 4 | GioiThieuNgan | text | Giới thiệu ngắn |
| 5 | BannerChuyenKhoa | varchar(5000) | Banner chuyên khoa |
| 6 | ThongTinGioiThieu | varchar(5000) | Thông tin giới thiệu |
| 7 | PhuongPhapDieuTri | text | Phương pháp điều trị |
| 8 | DichVu | text | Dịch vụ |
| 9 | DaXoa | bit(2) | Đã xoá |

**d. Bảng tbl_danhmuc**

Bảng tbl_ danhmuc dùng để lưu các thông tin danh mục của bài viết trong hệ thống của phòng khám như là tên danh mục, mã danh mục, đã xoá.

Bảng 2. 4 Bảng danh mục

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaDanhMuc | int | Mã danh mục, Khoá chính |
| 2 | TenDanhMuc | varchar(200) | Tên danh mục |
| 3 | DaXoa | bit(10) | Đã xoá |

**e. Bảng tbl_danhmuc_baidang**

Bảng tbl_ danhmuc_baidang dùng để lưu các thông tin của bài viết thuộc danh mục cụ thể trong hệ thống của phòng khám như là mã danh mục, mã bài đăng, đã xoá.

Bảng 2. 5 Bảng danh mục bài đăng

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaDanhMucBaiDang | int | Mã, Khoá chính |
| 2 | MaDanhMuc | int | Mã danh mục, khoá ngoại |
| 3 | MaBaiDang | int | Mã bài đăng, khoá ngoại |
| 4 | DaXoa | int | Đã xoá |

**f. Bảng tbl_dichvu**

Bảng tbl_baidang dùng để lưu các thông tin của bài viết trong hệ thống của phòng khám như là tiêu đề, mô tả ngắn, nội dung, ngày đăng,…

Bảng 2. 6 Bảng dịch vụ

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaDichVu | int | Mã dịch vụ, Khoá chính |
| 2 | TenDichVu | varchar(200) | Tên dịch vụ |
| 3 | MoTaNgan | text | Mô tả ngắn |
| 4 | AnhDichVu | varchar(5000) | Ảnh dịch vụ |
| 5 | TongQuan | text | Tổng quan |
| 6 | ThietBi | text | Thiết bị |
| 7 | QuyTrinh | text | Quy trình |
| 8 | GoiDichVu | text | Gói dịch vụ |
| 9 | KyThuatDieuTri | text | Kỹ thuật điều trị |
| 10 | DaXoa | bit(2) | Đã xoá |

**g. Bảng tbl_dichvu_bacsi**

Bảng tbl_ dichvu_bacsi dùng để lưu các thông tin của bác sĩ thuộc dịch vụ cụ thể trong hệ thống của phòng khám như là mã dịch vụ, mã bác sĩ, đã xoá.

Bảng 2. 7 Bảng dịch vụ bác sĩ

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaDichVuBacSi | int | Mã, Khoá chính |
| 2 | MaDichVu | int | Mã danh mục, khoá ngoại |
| 3 | MaBacSi | int | Mã bài đăng, khoá ngoại |
| 4 | DaXoa | int | Đã xoá |

**h. Bảng tbl_giới thiệu**

Bảng tbl_gioithieu dùng để lưu các thông tin giới thiệu của phòng khám như là ảnh giới thiệu, tầm nhìn, sứ mệnh, triết lý,…

Bảng 2. 8 Bảng giới thiệu

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaGioiThieu | int | Mã giới thiệu, Khoá chính |

| | | | |
|---|---|---|---|
| 2 | GioiThieuPhongKham | text | Giới thiệu phòng khám |
| 3 | AnhGioiThieuPhongKham | text | Ảnh giới thiệu |
| 4 | DoiNguBacSi | text | Đội ngũ bác sĩ |
| 5 | AnhDoiNguBacSi | text | Ảnh đội ngũ |
| 6 | TamNhin | text | Tầm nhìn |
| 7 | SuMenh | text | Sứ mệnh |
| 8 | TrietLy | text | Triết lý |
| 9 | GiaTriCotLoi | text | Giá trị cốt lõi |
| 10 | TrangThietBi | text | Trang thiết bị |
| 11 | AnhTrangThietBi | varchar(1000) | Ảnh trang thiết bị |
| 12 | DaXoa | bit(10) | Đã xoá |

**i. Bảng tbl_lichdatlich**

Bảng tbl_lichdatlich dùng để lưu các thông tin đặt lịch của khách hàng tại phòng khám như là mã bác sĩ, ngày đặt, tên bác sĩ, tên dịch vụ,…

Bảng 2. 9 Bảng lịch đặt lịch

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaLichDatLich | int | Mã, Khoá chính |
| 2 | MaBacSi | int | Mã bác sĩ, Khoá ngoại |
| 3 | MaNguoiDung | int | Mã người dùng, Khoá ngoại |
| 4 | NgayDat | date | Ngày đặt lịch |
| 5 | BuoiSang | int | Buổi sáng |
| 6 | BuoiTrua | int | Buổi trưa |
| 7 | BuoiChieu | int | Buổi chiều |
| 8 | TenDichVu | varchar(200) | Tên dịch vụ đặt |
| 9 | TenBacSi | varchar(200) | Tên bác sĩ đặt |
| 10 | VanDeSucKhoe | varchar(5000) | Vấn đề sức khoẻ |
| 11 | Sdt | varchar(100) | Số điện thoại |

| | | | |
|---|---|---|---|
| 12 | NgaySinh | date | Ngày sinh |
| 13 | HoTen | varchar(100) | Họ tên |
| 14 | GioiTinh | varchar(100) | Giới tính |
| 15 | DiaChi | varchar(500) | Địa chỉ |
| 16 | TrangThaiDat | int | Trạng thái đặt |
| 17 | ThoiGian | datetime | Thời gian đặt |
| 18 | DaXoa | int | Đã xoá |

**j. Bảng tbl_lichlamviec**

Bảng tbl_lichlamviec dùng để lưu thông tin thời gian trống của bác sĩ trong hệ thống của phòng khám như là ngày làm việc, mã bác sĩ, trạng thái sáng, trạng thái trưa, trạng thái chiều,…

Bảng 2. 10 Lịch làm việc

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaLichLamViec | int | Mã, Khoá chính |
| 2 | MaBacSi | int | Mã bác sĩ, Khoá ngoại |
| 3 | NgayLamViec | date | Ngày làm việc |
| 4 | TrangThaiSang | int | Trạng thái sáng |
| 5 | TrangThaiTrua | int | Trạng thái trưa |
| 6 | TrangThaiChieu | int | Trạng thái chiều |
| 7 | DaXoa | bit(10) | Đã xoá |

**k. Bảng tbl_lienhe**

Bảng tbl_lienhe dùng để lưu các thông tin liên hệ của phòng khám như tên phòng khám, địa chỉ, số điện thoại, email,…

Bảng 2. 11 Bảng liên hệ

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaLienHe | int | Mã liên hệ, Khoá chính |
| 2 | TenPhongKham | varchar(500) | Tên phòng khám |

| | | | |
|---|---|---|---|
| 3 | DiaChi | varchar(1000) | Địa chỉ |
| 4 | SoDienThoai | varchar(200) | Số điện thoại |
| 5 | Email | varchar(500) | Email |
| 6 | ThoiGianLamViec | varchar(500) | Thời gian làm viêc |
| 7 | DaXoa | bit(10) | Đã xoá |

**l. Bảng tbl_nguoidung**

Bảng tbl_nguoidung dùng để lưu thông tin người dùng trong hệ thống của phòng khám như là tài khoản, mật khẩu, tên người dùng,…

Bảng 2. 12 Bảng người dùng

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaNguoiDung | int | Mã, Khoá chính |
| 2 | TaiKhoan | varchar(200) | Tài khoản |
| 3 | MatKhau | varchar(2000) | Mật khẩu |
| 4 | TenNguoiDung | varchar(200) | Tên người dùng |
| 5 | Email | varchar(200) | Email |
| 6 | SoDienThoai | varchar(200) | Số điện thoại |
| 7 | NgaySinhNguoiDung | date | Ngày sinh |
| 8 | AnhNguoiDung | varchar(5000) | ảnh người dùng |
| 9 | LoaiNguoiDung | int | Loại người dùng |
| 10 | DaXoa | bit | Đã xoá |

**m. Bảng tbl_nhan**

Bảng tbl_ nhan dùng để lưu các thông tin của nhãn của các bài viết như là mã nhãn, tên nhãn, đã xoá.

Bảng 2. 13 Bảng nhãn

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaNhan | int | Mã, Khoá chính |
| 2 | TenNhan | varchar(200) | Tên nhãn |

| | | | |
|---|---|---|---|
| 3 | DaXoa | bit(10) | Đã xoá |

**n. Bảng tbl_nhan_baidang**

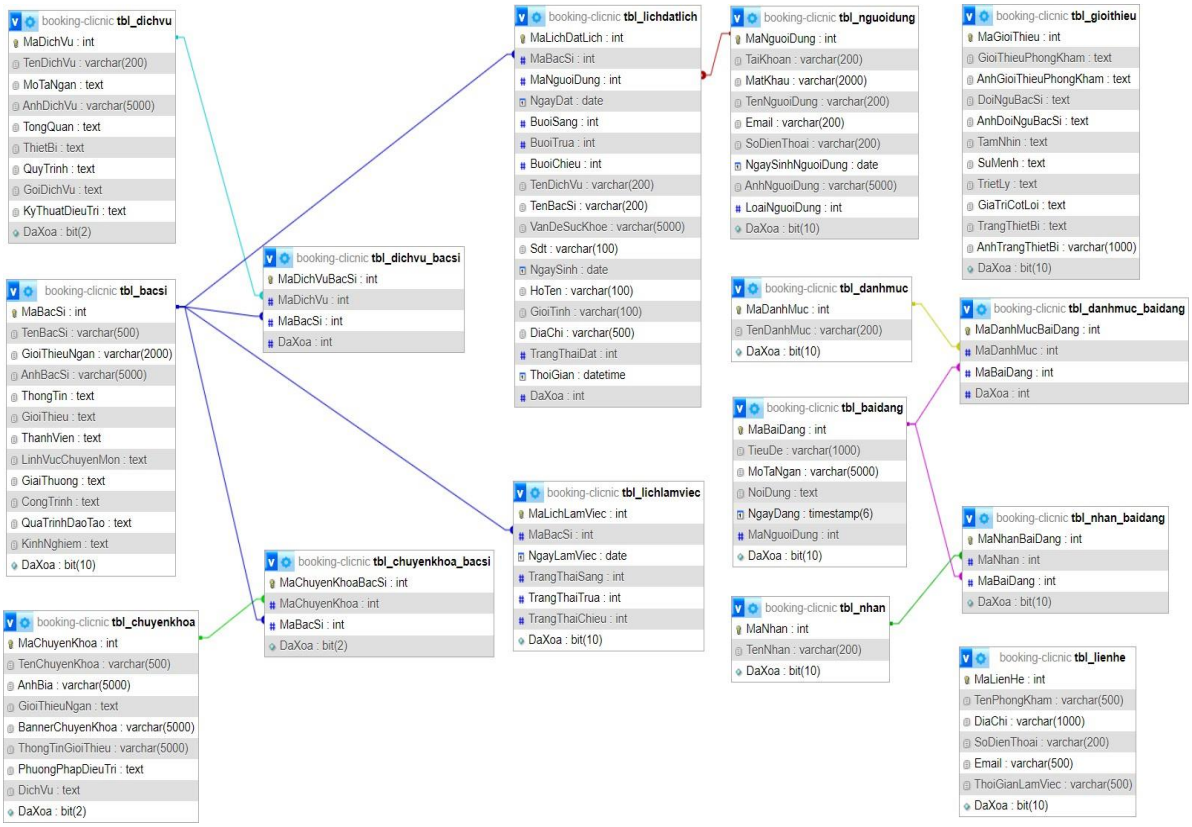
Bảng tbl_ nhan_baidang dùng để lưu các thông tin của bài viết thuộc nhãn như là mã nhãn, mã bài đăng, đã xoá.

Bảng 2. 14 Bảng nhãn bài đăng

| STT | Tên thuộc tính | Kiểu dữ liệu | Mô tả |
|---|---|---|---|
| 1 | MaNhanBaiDang | int | Mã, Khoá chính |
| 2 | MaNhan | int | Mã nhãn |
| 3 | MaBaiDang | int | Mã bài đăng |
| 4 | DaXoa | bit(10) | Đã xoá |

**o. Sơ đồ lớp hệ thống**

Trong sơ đồ lớp này, sẽ hiển đầy toàn bộ các thông tin của các bảng trong cơ sở dữ liệu các ràng buộc khoá chính, khoá ngoại như hình bên dưới




Hình 2. 22 Sơ đồ lớp

## 2.4 Một số giải thuật tiêu biểu

### 2.4.1 Giải thuật tìm kiếm theo tên bác sĩ

Mục tiêu: Tìm kiếm đúng và gần đúng tên bác sĩ với nội dung người dùng nhập.

Thuật toán sử dụng: Giải thuật tìm kiếm chuỗi thường sử dụng so sánh ký tự từng vị trí của từ khoá với dữ liệu trong tập dữ liệu.

Quá trình tìm kiếm:

- Nhập dữ liệu: Chuỗi từ khoá được nhập từ người dùng.
- Xử lý dữ liệu: Dữ liệu trong tập dữ liệu được duyệt qua để tìm kiếm chuỗi từ khoá.
- So sánh và đánh giá: So sánh từng ký tự của từ khoá với dữ liệu để xác định kết quả tìm kiếm.

Kết quả: trả về tên chính xác thông tin của bác sĩ hoặc một danh sách thông tin của các bác sĩ gần đúng.

Sử dụng giải thuật này giúp tối ưu hóa quá trình tìm kiếm của người dùng, thuận tiện cho người dùng tìm kiếm bác sĩ để đặt lịch.

### 2.4.2 Giải thuật lập lịch theo thời gian bác sĩ

Mục tiêu: tìm và xác định thời gian trống trong lịch làm việc của bác sĩ để hiển thị lịch cho người dùng đặt lịch khám.

Thuật toán sử dụng: giải thuật tìm kiếm trong lịch làm việc của bác sĩ để xác định thời gian trống dựa trên yêu cầu đặt lịch của người dùng.

Quá trình tìm kiếm:

- Nhập dữ liệu: người dùng là bác sĩ vào hệ thống thiết lập thời gian trống của bản thân thông qua giao diện như là ngày và buổi trống.
- Xử lý dữ liệu: dữ liệu lịch trống của bác sĩ sẽ được xử lý để hiển thị lên giao diện của người dùng để người dùng đặt lịch
- Lập lịch và xác nhận: gửi xác nhận lịch hẹn và cập nhật lịch làm việc của bác sĩ.

Kết quả: hiển thị thông tin chi tiết về cuộc hẹn, bao gồm cả thời gian, địa điểm, và thông tin của bác sĩ.

Sử dụng giải thuật này giúp tối ưu hóa quá trình đặt lịch khám theo thời gian trống của bác sĩ và cung cấp một trải nghiệm thuận tiện cho cả bác sĩ và bệnh nhân.

## 2.5 Phát triển hệ thống

### 2.5.1 Môi trường phát triển

Ứng dụng được xây dựng trên nền tảng Node.js, và sử dụng Express Framework để phát triển. Express là một framework Node.js mạnh mẽ giúp tạo ra các ứng dụng web và API một cách nhanh chóng và hiệu quả.

Ứng dụng được xây dựng theo mô hình client-server, mang lại lợi ích đáng kể trong việc đơn giản hóa quá trình phát triển và tối ưu hóa hiệu suất. Việc triển khai mô hình client-server giúp tách biệt logic và chức năng giữa phía client và phía server. Phía client chịu trách nhiệm về giao diện người dùng và tương tác người dùng, trong khi phía server xử lý logic, lưu trữ dữ liệu và thực hiện các tác vụ chức năng. Lợi ích của mô hình client-server bao gồm sự linh hoạt trong việc phát triển và bảo trì, vì có thể thay đổi và cập nhật client mà không ảnh hưởng đến server và ngược lại.

Công cụ phát triển ứng dụng: Visual Studio Code 2022 công cụ phát triển chính của ứng dụng.

Hệ quản trị cơ sở dữ liệu của ứng dụng: MySql

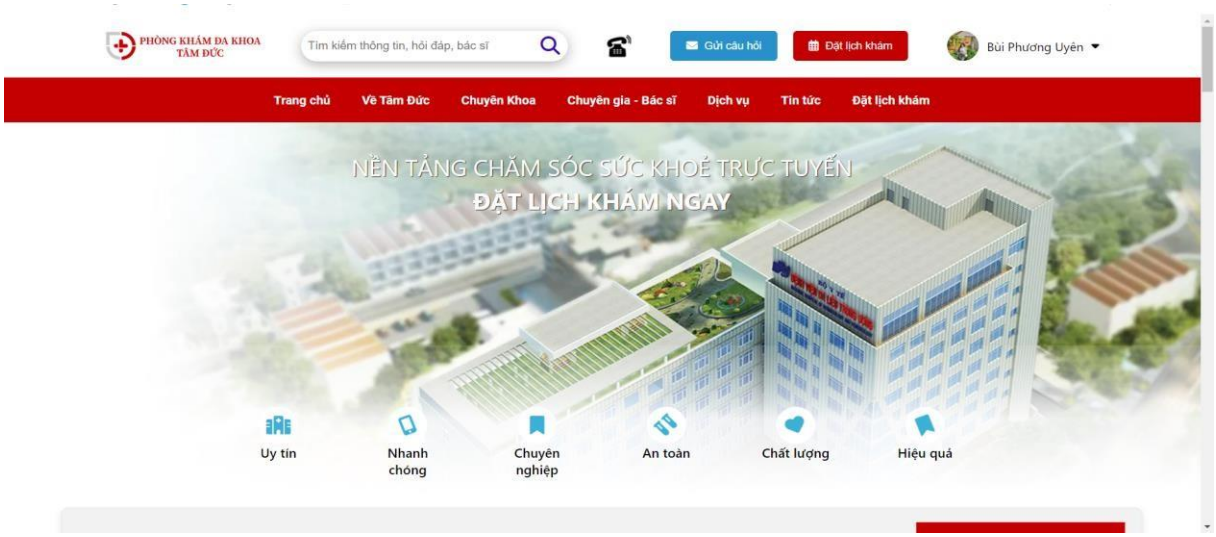
Công nghệ sử dụng phía client: ReactJs để xây dựng giao diện, REST để truyền thông dữ liệu giữa client và server, React Router để quản lý định tuyến.

Công nghệ sử dụng phía server: ExpressJs để xây dựng API hệ thống.

### 2.5.2 Một số giao diện chính

#### 2.5.2.1 Giao diện trang chủ

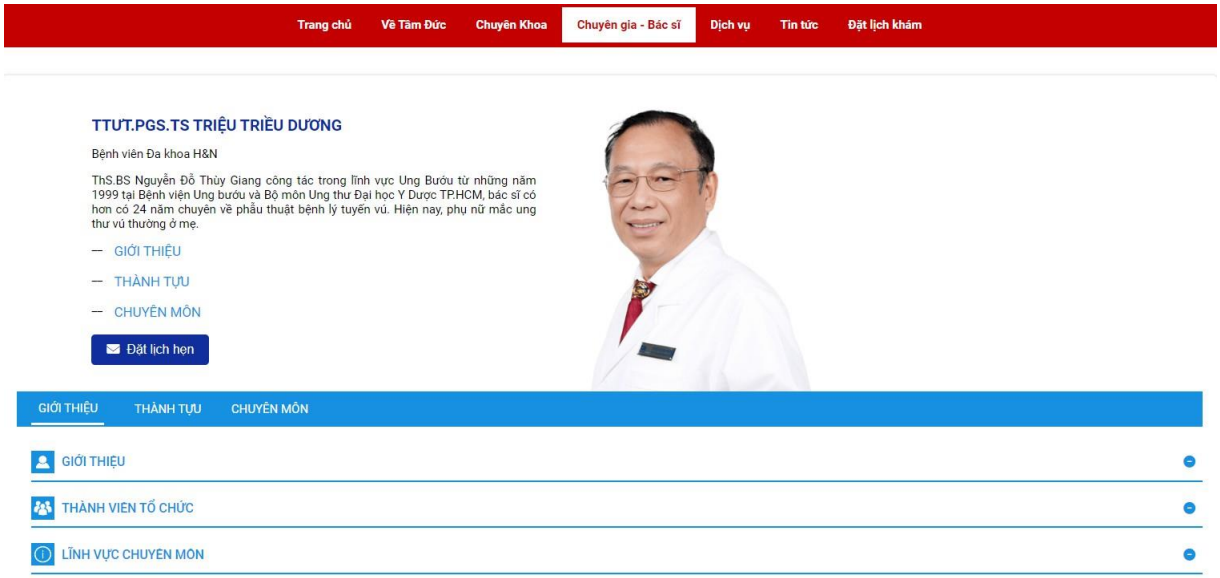
Khi đăng nhập thành công, người dùng sẽ được chuyển đến trang chủ, tại đây người dùng có thể sử dụng các chức năng tại trang như xem chuyên khoa, xem dịch vụ, đặt lịch,…




Hình 2. 23 Giao diện trang chủ

### 2.5.2.2 Giao diện chi tiết chuyên gia

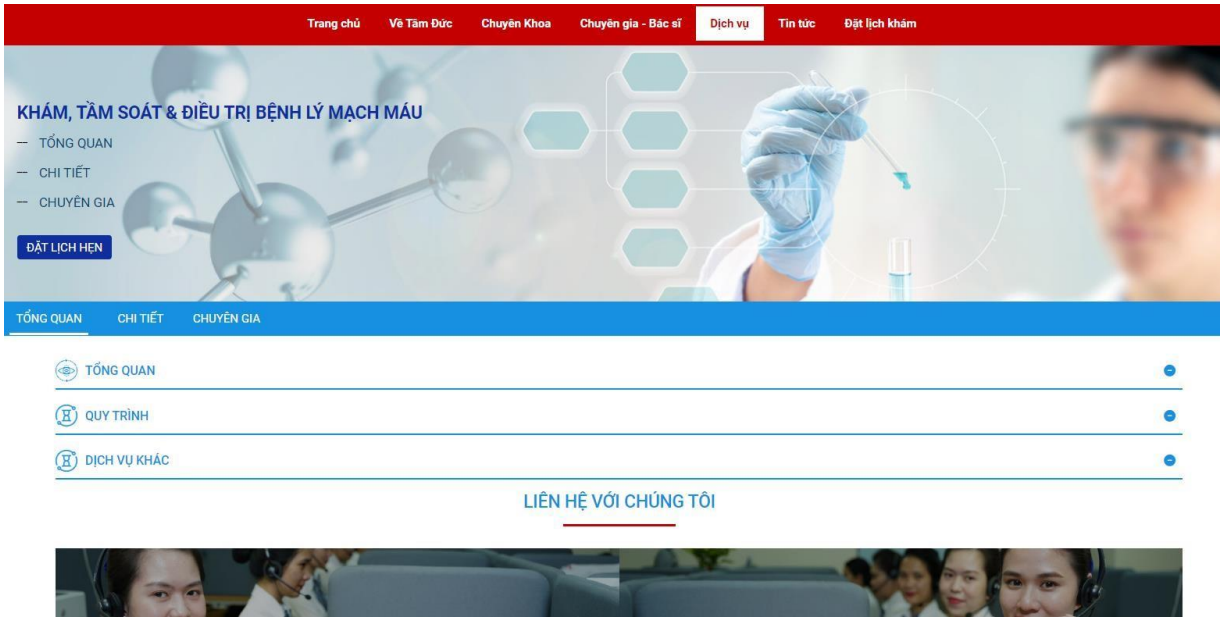
Khi người dùng truy cập vào một bác sĩ bất kì, người dùng sẽ xem được đầy đủ các thông tin của bác sĩ này như là thành viên tổ chức, lĩnh vực chuyên môn,… Bên cạnh đó người dùng còn có thể đặt lịch trực tiếp với bác sĩ thông qua nút "đặt lịch hẹn" để có thể đặt lịch nhanh với bác sĩ.




Hình 2. 24 Giao diện chi tiết bác sĩ

### 2.5.2.3 Giao diện chi tiết dịch vụ

Khi người dùng chọn chi tiết một dịch vụ bất kì, trang chi tiết dịch vụ sẽ hiển thị, tại đây người dùng có thể xem đầy đủ các thông tin của dịch vụ như là tổng quan, quy trình,… Bên cạnh đó, người dùng còn có thể xem các bác sĩ thuộc dịch vụ này để người dùng thuận tiện đặt lịch trong quá trình xem.




Hình 2. 25 Giao diện chi tiết dịch vụ

### 2.5.2.4 Giao diện tin tức

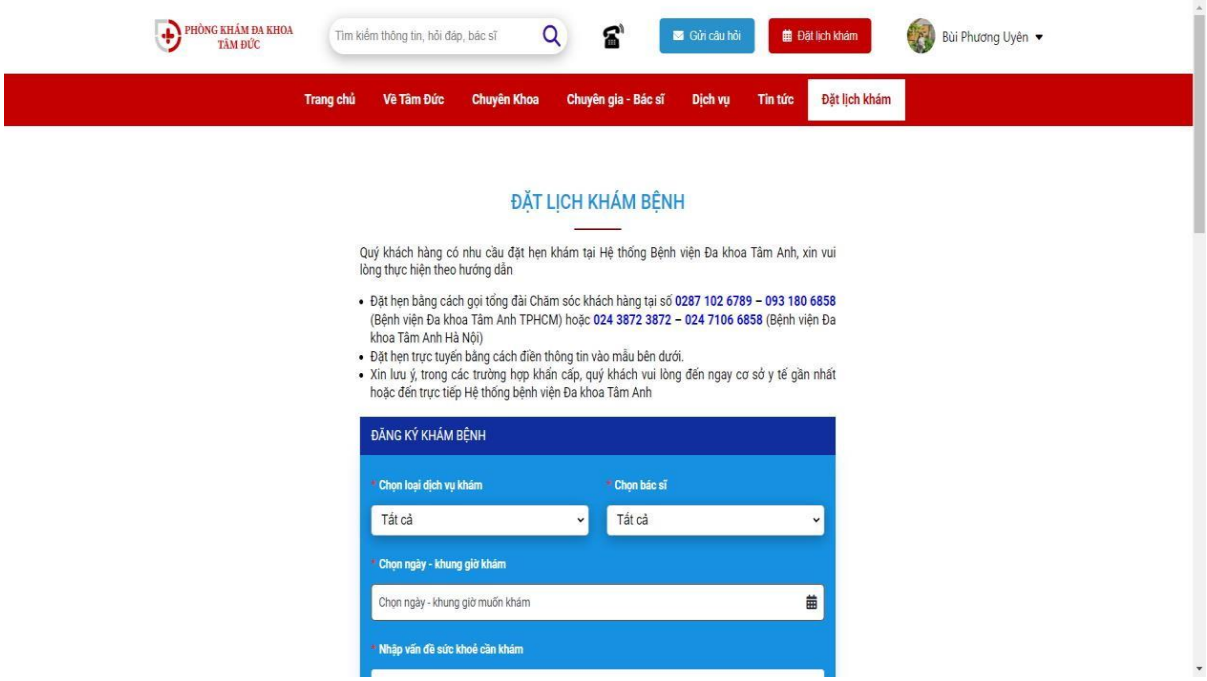
Tại giao diện bài viết người dùng có thể xem các bài viết mới nhất, các hoạt động của phòng khám, các mẹo, tiện ích được phòng khám chia sẻ như ảnh bên dưới.




Hình 2. 26 Giao diện tin tức

### 2.5.2.5 Giao diện đặt lịch khám

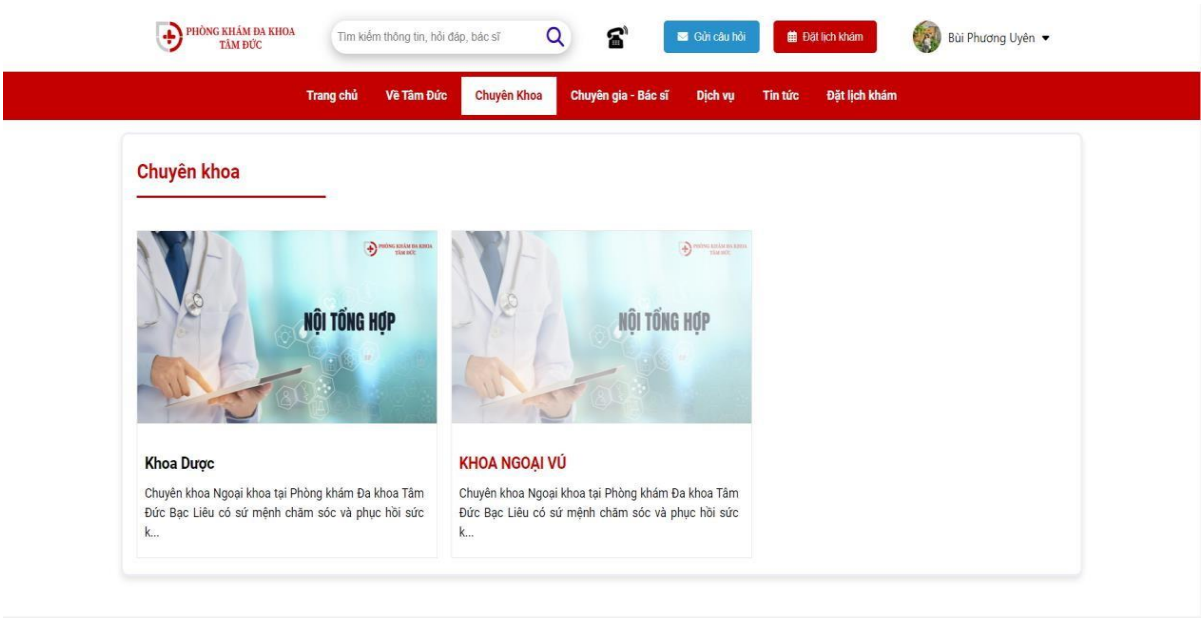
Khi người dùng chọn chức năng đặt lịch khám, người dùng sẽ thấy một biểu mẫu đăng kí như ảnh bên dưới. Tại đây người dùng cần nhập đầy đủ các thông tin như loại dịch vụ, bác sĩ, khung giờ khám và một số thông tin cá nhân để hoàn thành đặt lịch.




Hình 2. 27 Giao diện đặt lịch khám

### 2.5.2.6 Giao diện trang chuyên khoa

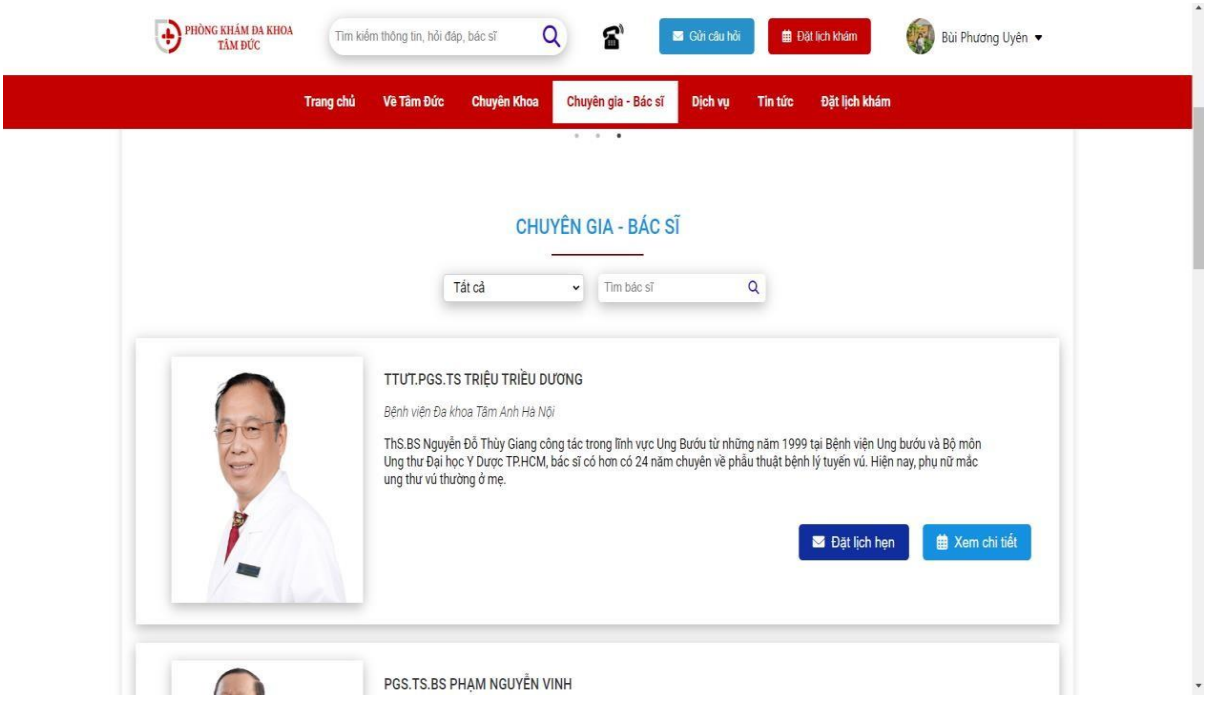
Khi người dùng chuyên khoa trên thanh menu, người dùng sẽ được chuyển đến trang chuyên khoa. Tại đây người dùng sẽ thấy một danh sách các chuyên khoa cho người dùng lựa chọn như hình bên dưới.




Hình 2. 28 Giao diện trang chuyên khoa

### 2.5.2.7 Giao diện trang chuyên gia

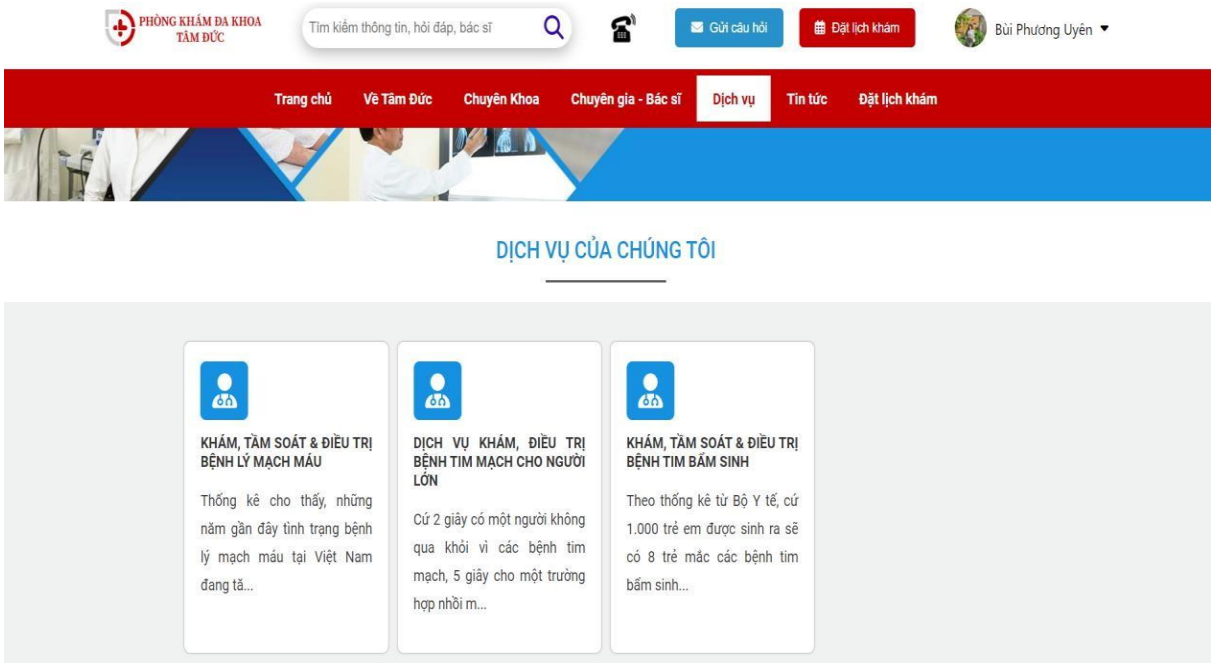
Khi người dùng chọn menu chuyên gia bác sĩ, người dùng sẽ được chuyển đến trang chuyên gia bác sĩ. Tại đây người dùng xem được danh sách các bác sĩ, thông tin ngắn, tìm kiếm bác sĩ và lọc bác sĩ theo chuyên khoa.




Hình 2. 29 Giao diện trang chuyên gia

### 2.5.2.8 Giao diện trang dịch vụ

Khi người dùng chọn dịch vụ tại menu, người dùng sẽ chuyển đến trang dịch vụ. Tại đây người dùng xem được danh sách các dịch vụ mà phòng khám cung cấp như ảnh bên dưới.




Hình 2. 30 Giao diện trang dịch vụ

### 2.5.2.9 Giao diện trang giới thiệu

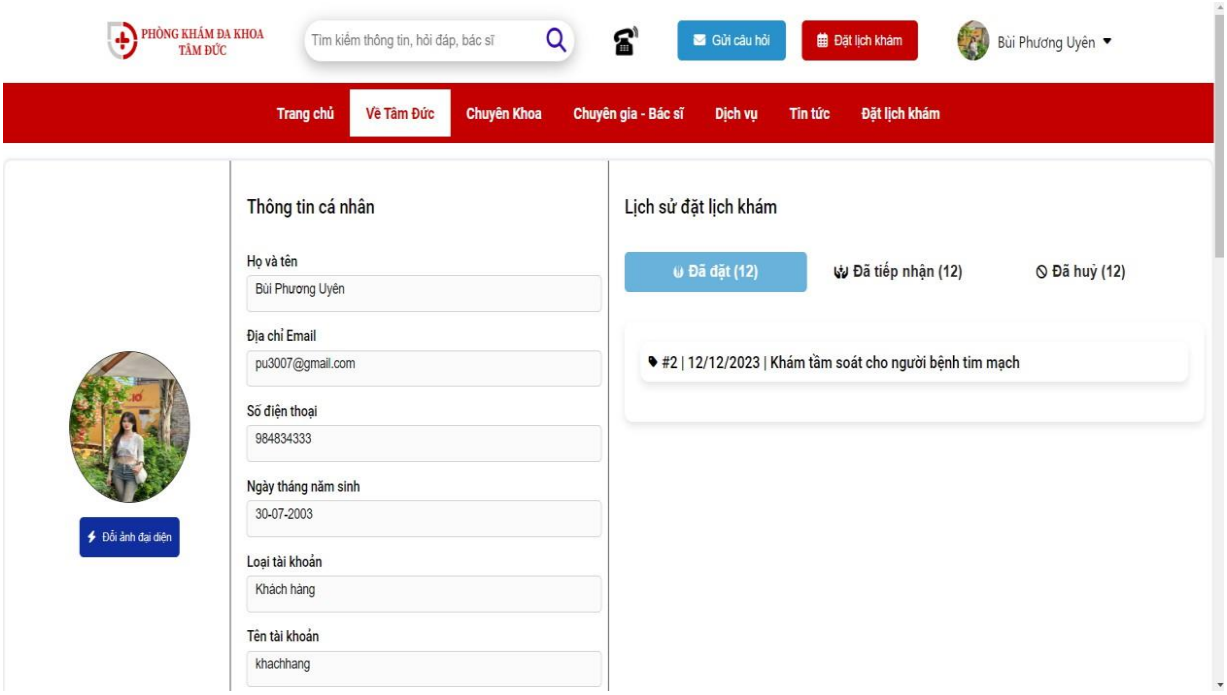
Tại trang giới thiệu, người dùng sẽ được xem các thông tin giới thiệu phòng khám như là sứ mệnh, triết lý,… như ảnh bên dưới.




Hình 2. 31 Giao diện trang giới thiệu

### 2.5.2.10 Giao diện trang thông tin cá nhân

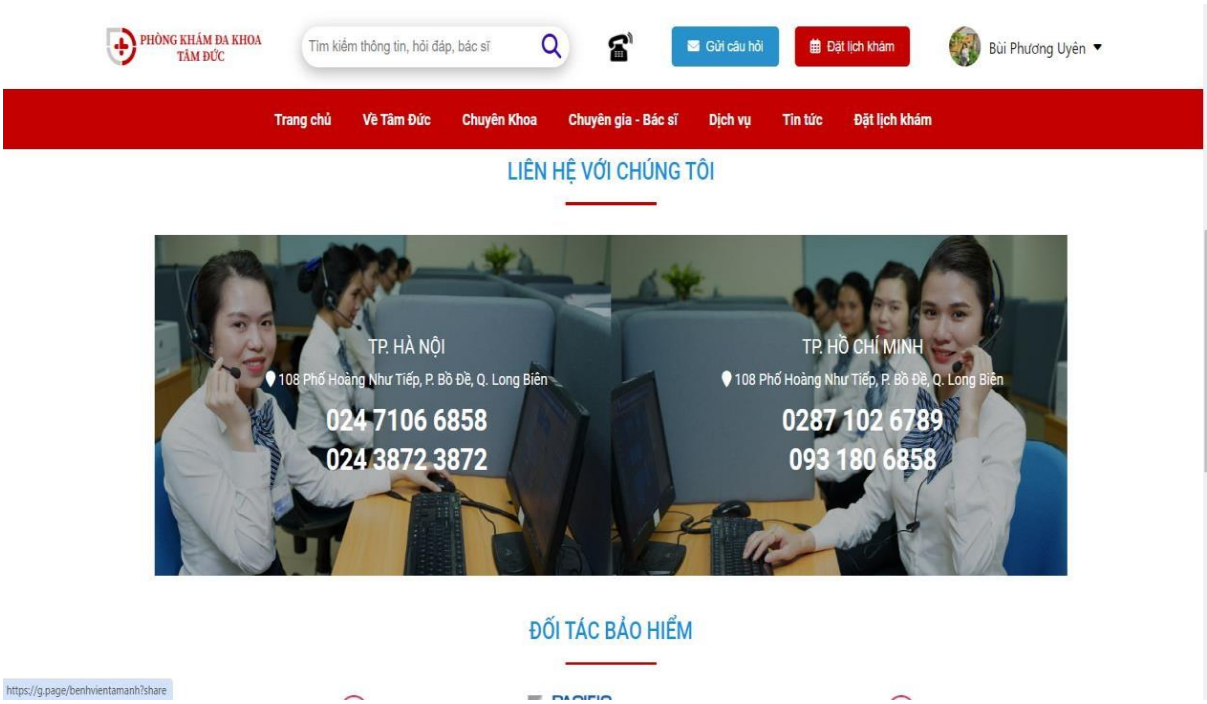
Khi người dùng đã đăng nhập vào hệ thống, người dùng sẽ được xem đầy đủ các thông tin cá nhân và lịch sử đặt lịch khám như ảnh bên dưới.




Hình 2. 32 Giao diện trang thông tin cá nhân

### 2.5.2.11 Giao diện thông tin liên hệ

Thông tin liên hệ như ảnh bên dưới được gắn vào hầu hết các trang trong hệ thống, người dùng chỉ cần kéo xuống đến gần cuối thì sẽ có thể thấy. Thông tin gồm địa chỉ, số điện thoại liên lạc của phòng khám.




Hình 2. 33 Giao diện thông tin liên hệ

# PHẦN BA: PHẦN KẾT LUẬN

--- oOo ---

## 3.1 Kết quả đạt được

Qua thời gian nghiên cứu tìm hiểu công nghệ và viết ứng dụng, chúng em đã đạt được một số kết quả nhất định. Hoàn thành được mục tiêu xây dựng website đặt lịch khám bệnh tại phòng khám Hoàng Nam. Các chức năng đã đáp ứng được yêu cầu đặt ra ban đầu, chúng em đã nghiên cứu và áp dụng các công nghệ và thư viện để hoàn thành ứng dụng này.

Những chức năng cụ thể mà chúng em đã thực hiện được như sau:

Người dùng là khách hàng: có thể đặt lịch khám theo lịch trống của bác sĩ, theo dõi trạng thái đặt lịch, xem thông tin bác sĩ, thông tin chuyên khoa, thông tin dịch vụ,…

Người dùng là bác sĩ: có thể cập nhật lịch trống để cho người dùng đặt lịch, xem danh sách khách hàng đặt lịch, theo dõi trạng thái đơn đặt lịch.

Người dùng là quản trị hệ thống: quản trị hệ thống, quản lý người dùng hệ thống (đăng ký, chỉnh sửa, xóa người dùng), quản lý vai trò gồm: khách hàng, bác sĩ, quản trị viên (phân quyền tài khoản hệ thống), quản lý chuyên khoa, quản lý dịch vụ, quản lý chuyên gia,...

Bên cạnh đó, hiểu được nghiệp vụ, phân tích đề tài cụ thể và thiết kế cơ sở dữ liệu.

Đồng thời, xây dựng hệ thống thông qua các sơ đồ usecase, sơ đồ hoạt động dựa trên bộ công cụ StarUML.

## 3.2 Hạn chế

Giao diện chưa chuyên nghiệp, các bố cục, màu chưa phù hợp.

Ứng dụng chưa thích hợp với đa dạng thiết bị như điện thoại, máy tính bảng.

Chưa đầy đủ chức năng của một ứng dụng.

## 3.3 Hướng phát triển

Tối ưu hóa tốc độ để tăng trải nghiệm người dùng.

Hoàn thiện, bổ sung một số chức năng còn thiếu để đáp ứng yêu cầu nghiệp vụ của đề tài.

Xây dựng giao diện thích hợp với nhiều thiết bị.

Thiết kế giao diện bắt mắt, thích hợp với người dùng để tăng trải nghiệm người dùng hệ thống.

# TÀI LIỆU THAM KHẢO

## I. Giáo trình, bài giảng

[1] Nguyễn Đình Thuân, Mai Xuân Hùng, "Giáo trình phát triển ứng dụng web", ĐHQG TPHCM, 2015

[2] Phạm Nguyễn Cương, Hồ Tường Vinh, "Giáo trình Phân tích và thiết kế hệ thống Hướng đối tượng sử dụng UML", ĐHKHTN TP HCM, 2013.

[3] …….

## II. Website

[1] https://aptech.fpt.edu.vn/nodejs-la-gi.html

[2] https://topdev.vn/blog/express-js-la-gi/

[3] https://topdev.vn/blog/api-la-gi/

[4] https://tenten.vn/tin-tuc/xampp-la-gi/

[5] https://200lab.io/blog/reactjs-la-gi/

[6] https://mona.media/mysql-la-gi/

[7] https://fct-club.com/2022/09/20/tong-quan-ve-bieu-do-uml/

[8] https://iviettech.vn/blog/302-ooaduml.html